

CS2109S: Introduction to AI and Machine Learning

Lecture 2: Solving Problems by Searching

20 August 2026

[PolLEV.com/rizki](https://polly.com/rizki)

DO NOT CLOSE YOUR POLLEVERYWHERE APP

There will be activities ahead

For those who are just enrolled

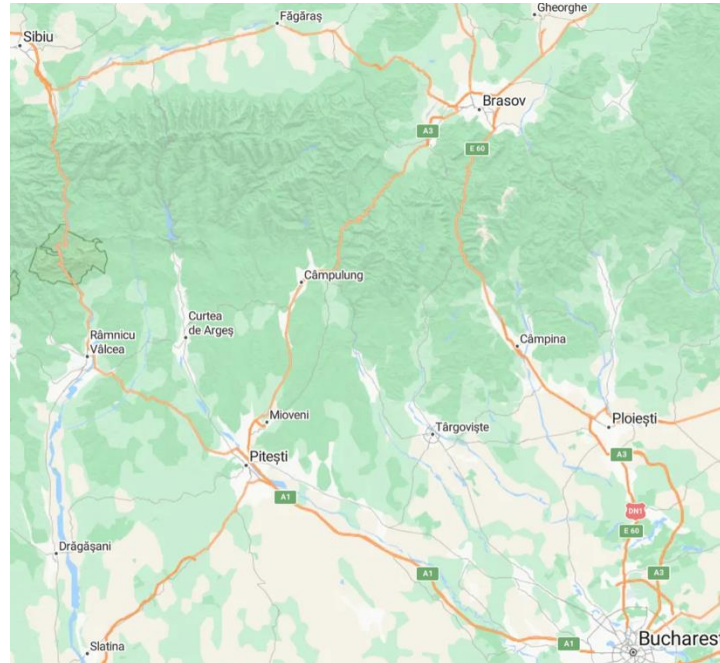
- Please watch the recording or read lecture slides for Lecture 1
- **Important:** Course Policies (also available in Canvas)

Outline

- **Designing an agent for search problems**
- Search algorithms primer
- Uninformed search
 - Breadth-first, depth-first, and uniform-cost search
- Informed search
 - A* search
 - Heuristics
- Search strategies
 - Depth-limited and iterative-deepening search

Preliminary: Search Problems

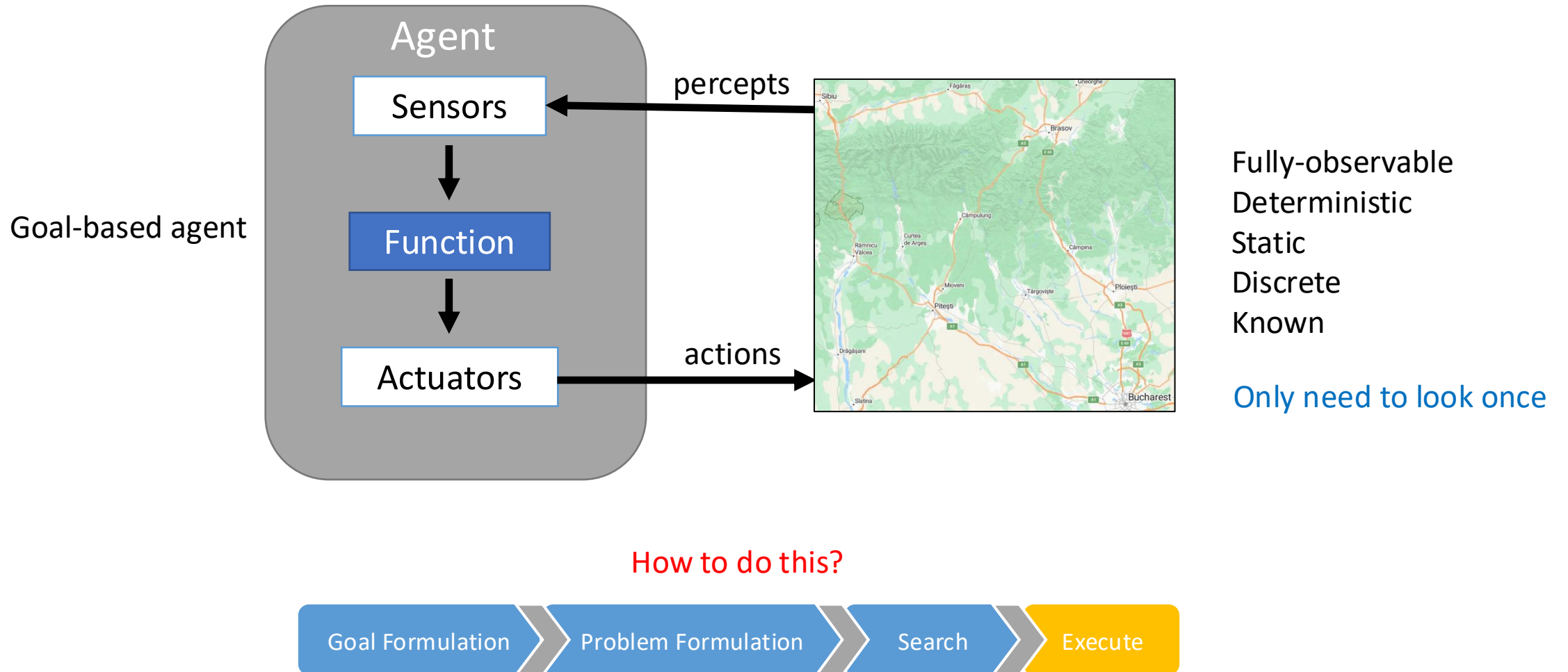
- A **search problem** refers to a type of problem where the goal is to find a state (or a path to a state) from a set of possible states by exploring various possibilities.
- Examples:
 - Path finding
 - Puzzle solving



9	1	3				5		
6		7					2	4
	5			8			7	
	7	9						
		2		9			4	3
					4		9	
	4				1	9		
7		6			9			5
		1			6	4		7

Designing an Agent

for problems that can be solved by **searching**. E.g., path finding



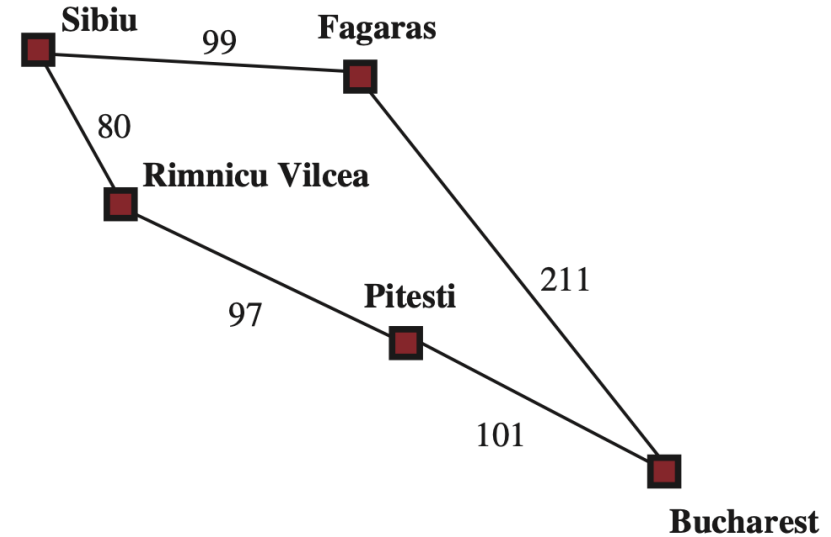
Example: Romania

Goal Formulation

Problem Formulation

Search

Execute



Problem Formulation

- States
- Initial state
- Goal state(s)/test
- Actions
- Transition model
- Action cost function

Cities: {at Sibiu, at Pitești, ...}

at Sibiu

at Bucharest

Go to {Fagaras, ...}

$T(\text{Sibiu, Go to Fagaras}) = \text{at Fagaras}$

See edges

Outline

- Designing an agent for search problems
- **Search algorithms primer**
- Uninformed search
 - Breadth-first, depth-first, and uniform-cost search
- Informed search
 - A* search
 - Heuristics
- Search strategies
 - Depth-limited and iterative-deepening search

Review: Data structures

Required operations

- Add element (“add”)
- Remove element (“pop”)
- Is empty



Credit: freepik.com

Queue

First-in-first-out (FIFO)



Credit: freepik.com

Priority Queue

High-priority-first-out



Credit: iStock

Stack

Last-in-first-out (LIFO)

Search

AIAMA: tree search

Goal Formulation

Problem Formulation

Search

Execute

create **frontier**

insert **Node**(initial_state) to **frontier**

while **frontier** is not empty:

node = **frontier**.pop()

 if **node.state** is goal: return solution

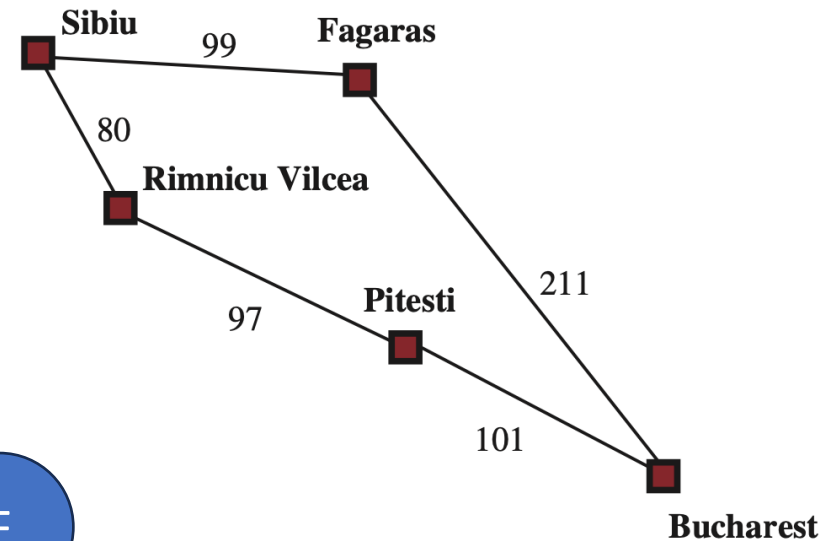
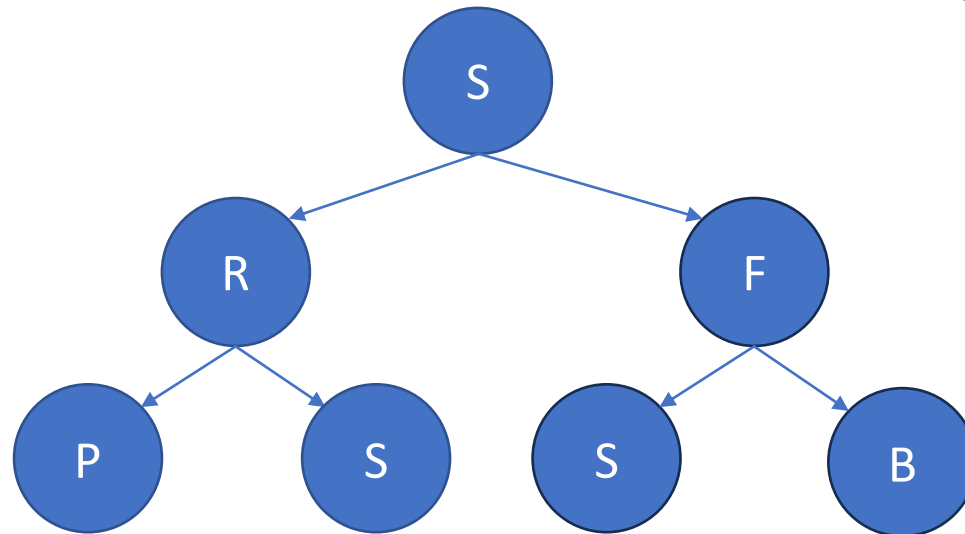
 for **action** in actions(**node.state**):

next_state = transition(**node.state**, **action**)

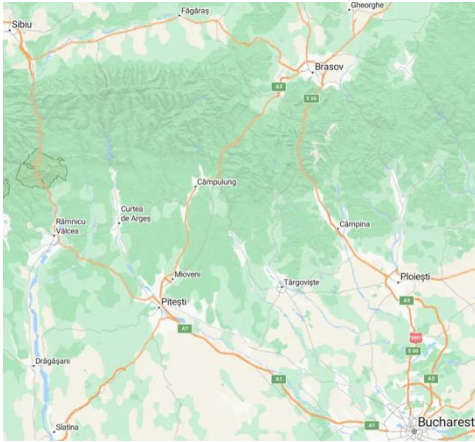
frontier.add(**Node**(**next_state**))

return failure

Can be queue, priority queue, stack
Frontier defines the search



Search Terminology (1)



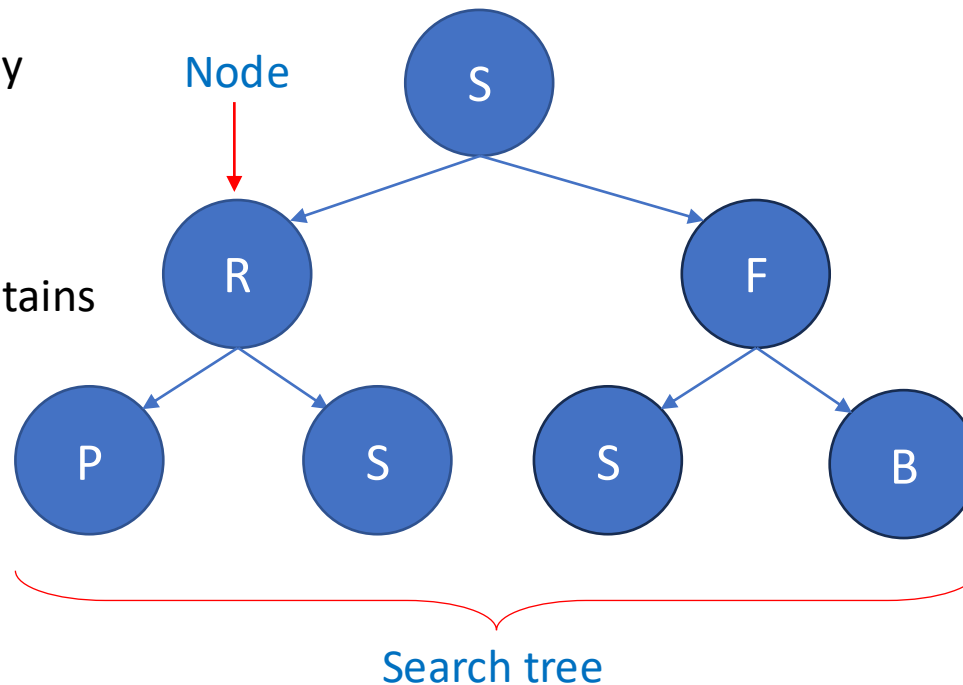
- A sequence of actions forms a **path**.
- A **solution** is a path from the initial state to a goal state.
- An **optimal solution** has the lowest path cost among all solutions.
- In some problems, a solution may simply be a desired state.
 - E.g., sudoku puzzle
- In this case, the notion of optimal solution is irrelevant.
- A search algorithm takes in a search problem as input and returns a **solution** / **failure**.

9	1	3				5		
6		7					2	4
	5			8			7	
	7	9						
		2		9			4	3
					4		9	
	4				1	9		
7		6			9			5
		1			6	4		7

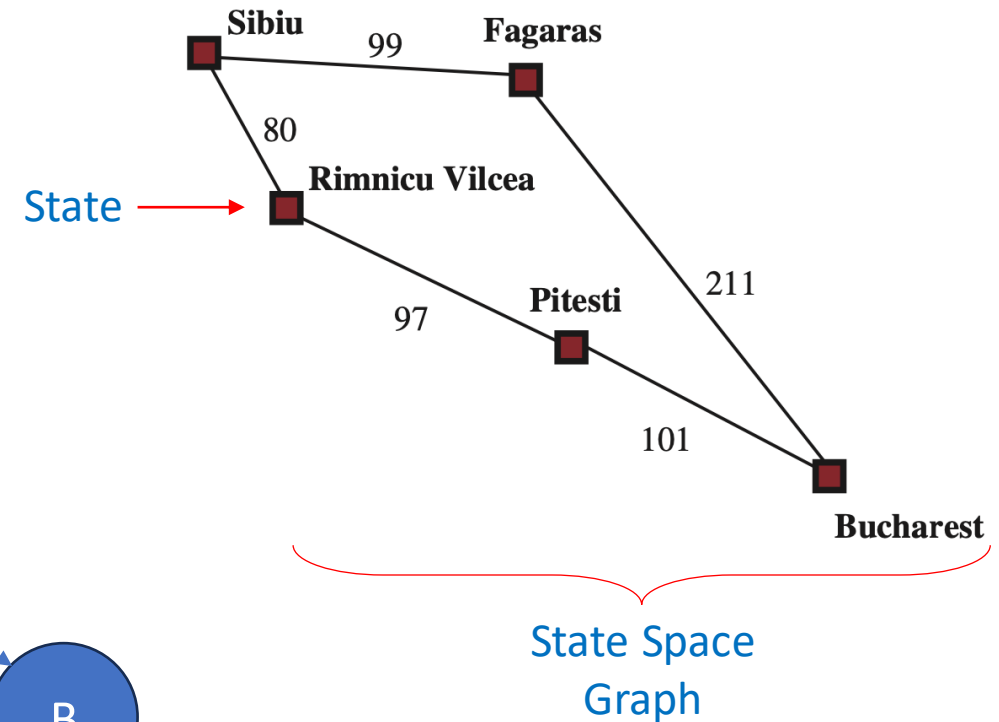
Search Terminology (2)

A search algorithm implicitly builds a **search tree** when exploring possible paths.

A **node** in a search tree contains a state in the state space.



The state space can be represented as a **graph** in which the vertices are states and the directed edges between them are actions.



Evaluation Criteria

(Worst-case) Complexity:

- **Time complexity**: number of nodes generated or expanded
- **Space complexity**: maximum number of nodes in memory
- In terms of branching factor, optimal depth, maximum depth, ...

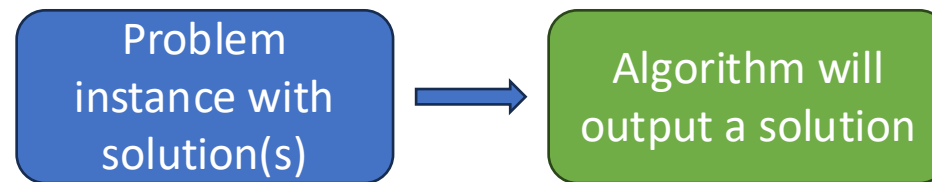
Disclaimer: CS2109S is not an algorithm-oriented course, so we will not discuss these complexities in detail

Others:

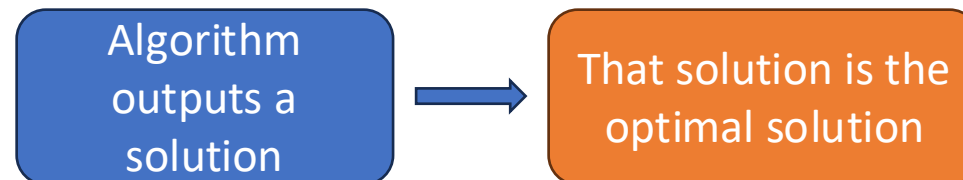
- **Completeness**
- **Optimality**

Completeness and Optimality (1)

- **Complete:** An algorithm is complete if, for every problem instance, it will find a solution if one exists. Conversely, an algorithm is incomplete if there exists at least one instance with a solution that the algorithm fails to find.



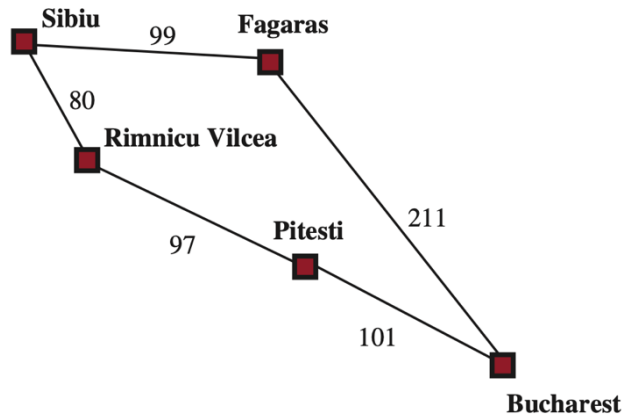
- **Optimal:** An algorithm is optimal if, for every instance where it produces a solution, that solution is the best possible. In other words, if the algorithm outputs a solution, it is guaranteed to be optimal.



Completeness and Optimality (2)

Common question: Are there optimal and incomplete algorithms?

- **Optimal and Incomplete:** An algorithm can be both optimal and incomplete. This means that there exists at least one instance where the algorithm fails to find a solution, even though, when it does find a solution, that solution is guaranteed to be optimal.

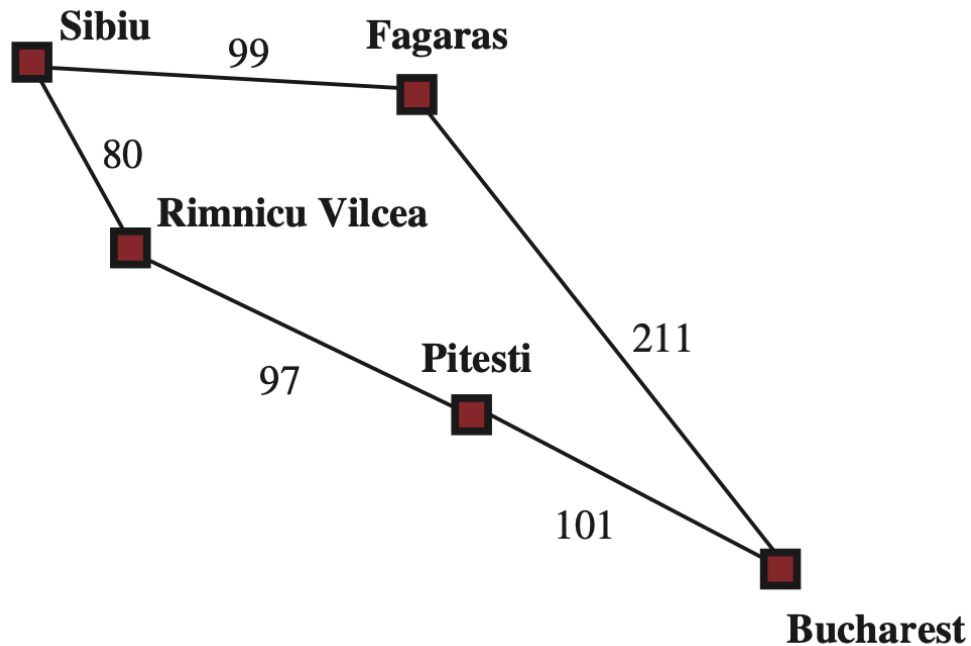


```
def search(state):  
    if state == "Fagaras":  
        return "go to Bucharest"  
    return None
```

Outline

- Designing an agent for search problems
- Search algorithms primer
- **Uninformed search**
 - Breadth-first, depth-first, and uniform-cost search
- Informed search
 - A* search
 - Heuristics
- Search strategies
 - Depth-limited and iterative-deepening search

Uninformed Search Algorithms



- The search algorithms is given no clue about how close a state is to the goal(s).
- E.g., an uninformed agent trying to reach Bucharest will not know whether going to Fagaras or Rimnicu Vilcea is a better step.

Search

AIAMA: tree search

```
create frontier
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

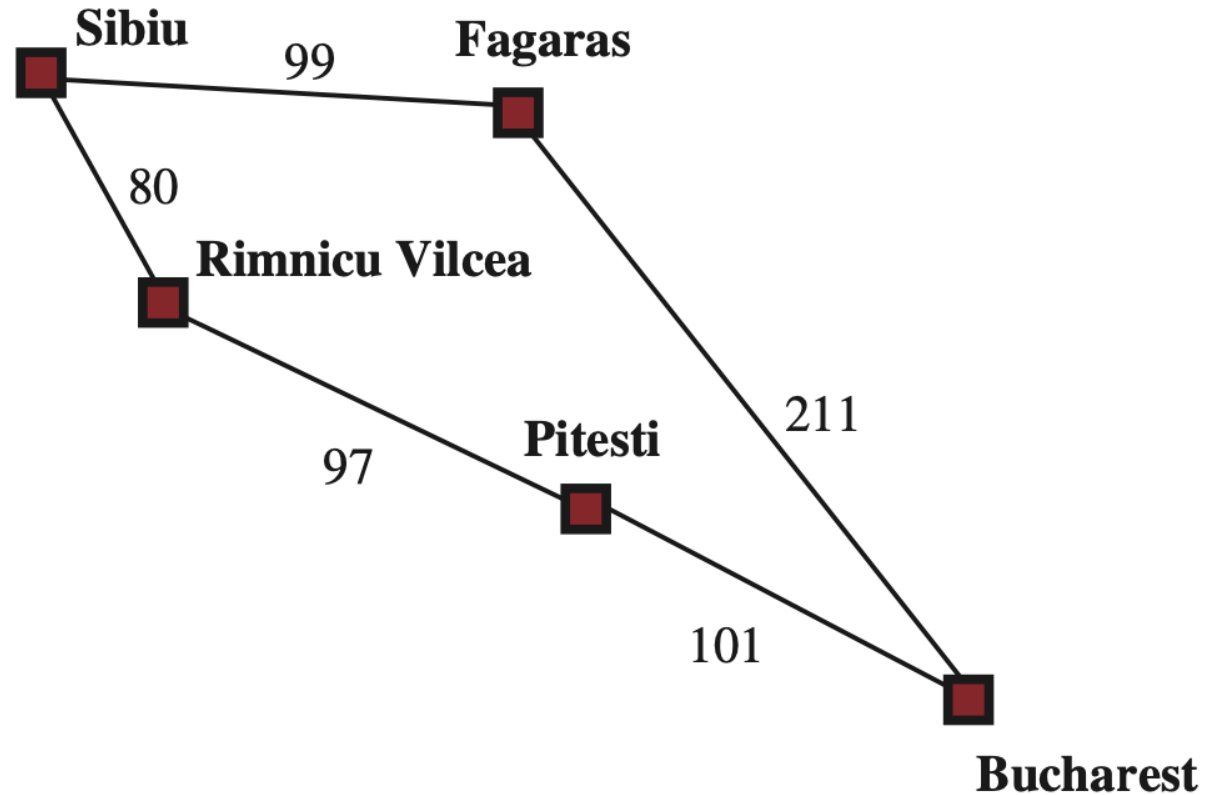
```
    if node.state is goal: return solution
```

```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```



Breadth-first Search (BFS)

```
create frontier : queue
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

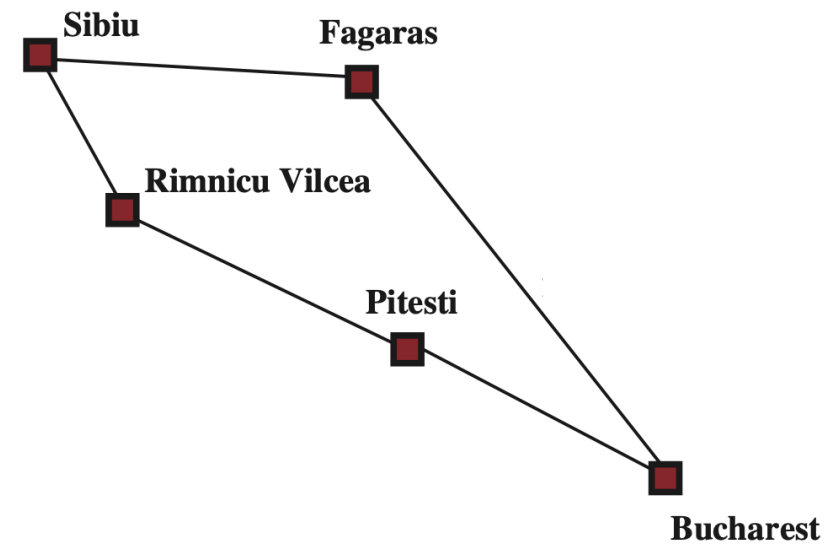
```
    if node.state is goal: return solution
```

```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```



Breadth-first Search (BFS)

```
create frontier : queue
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

```
    if node.state is goal: return solution
```

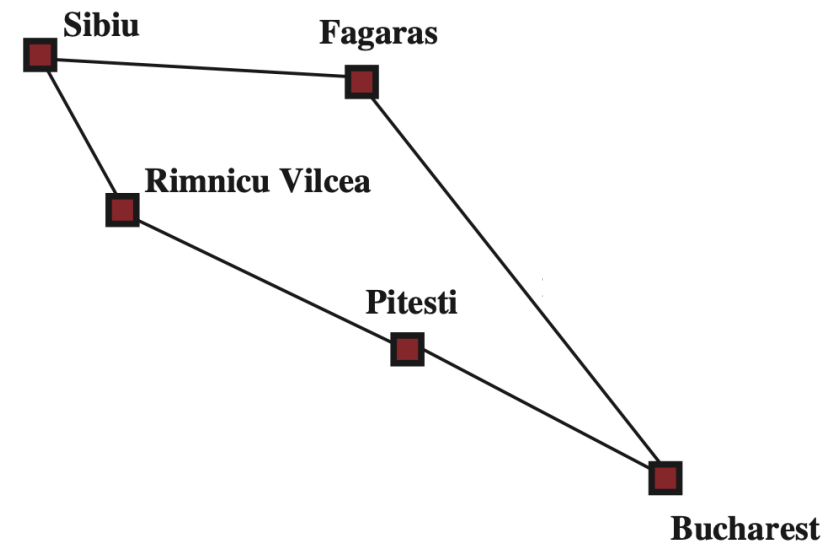
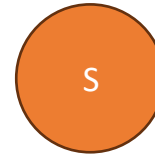
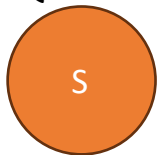
```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```

Queue:



Breadth-first Search (BFS)

```
create frontier : queue
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

```
    if node.state is goal: return solution
```

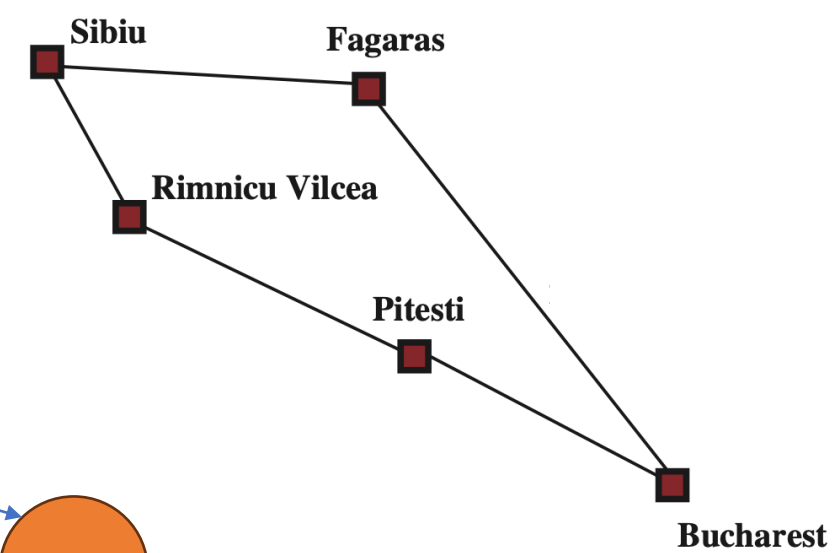
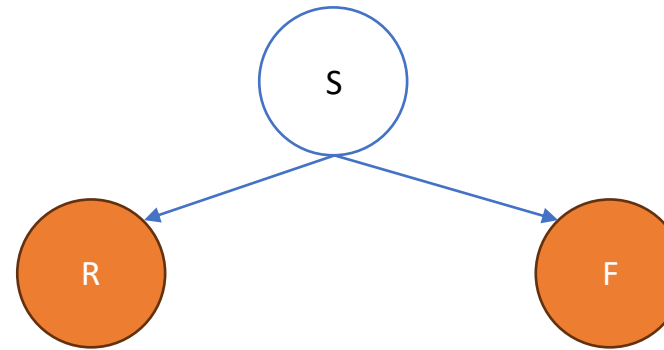
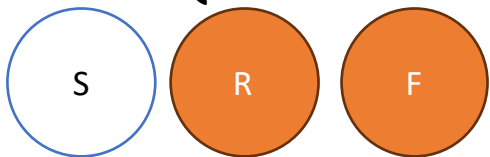
```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```

Queue:



Breadth-first Search (BFS)

```
create frontier : queue
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

```
    if node.state is goal: return solution
```

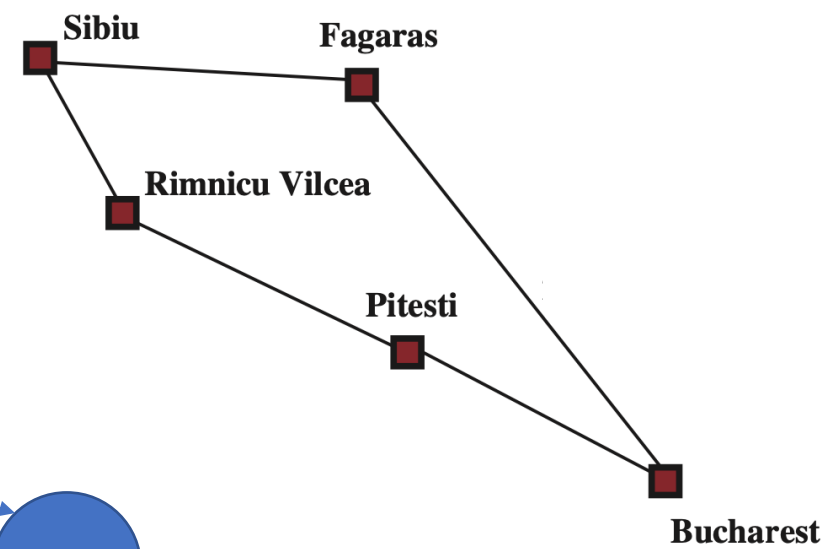
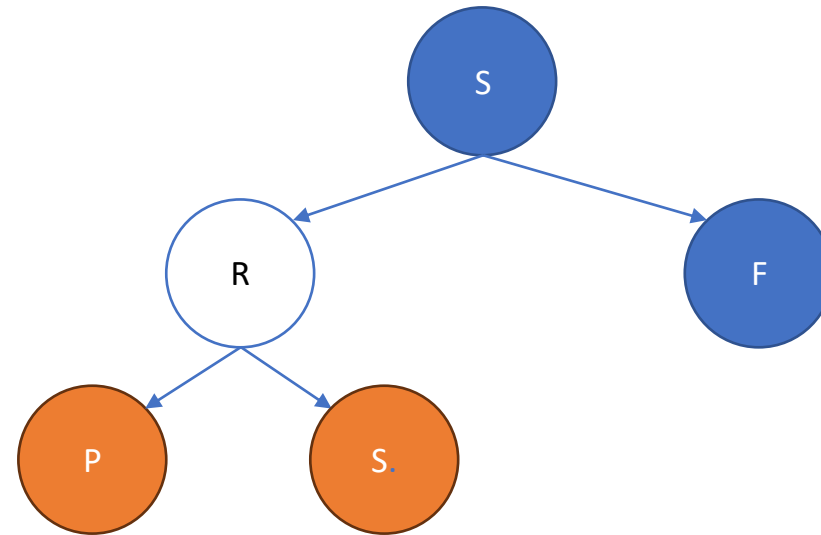
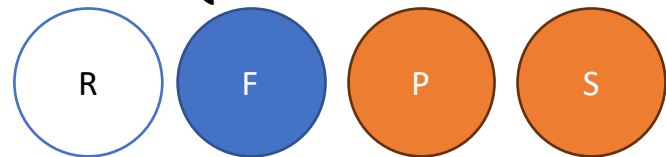
```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```

Queue:



Breadth-first Search (BFS)

```
create frontier : queue
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

```
    if node.state is goal: return solution
```

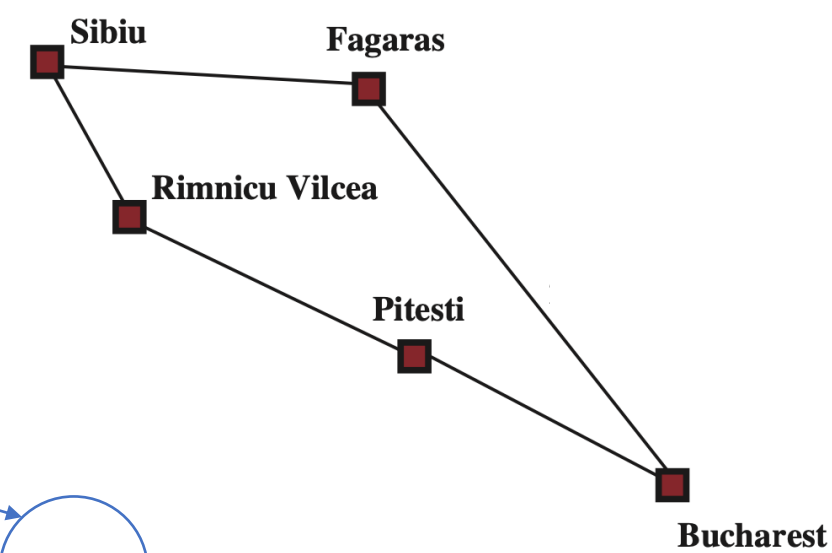
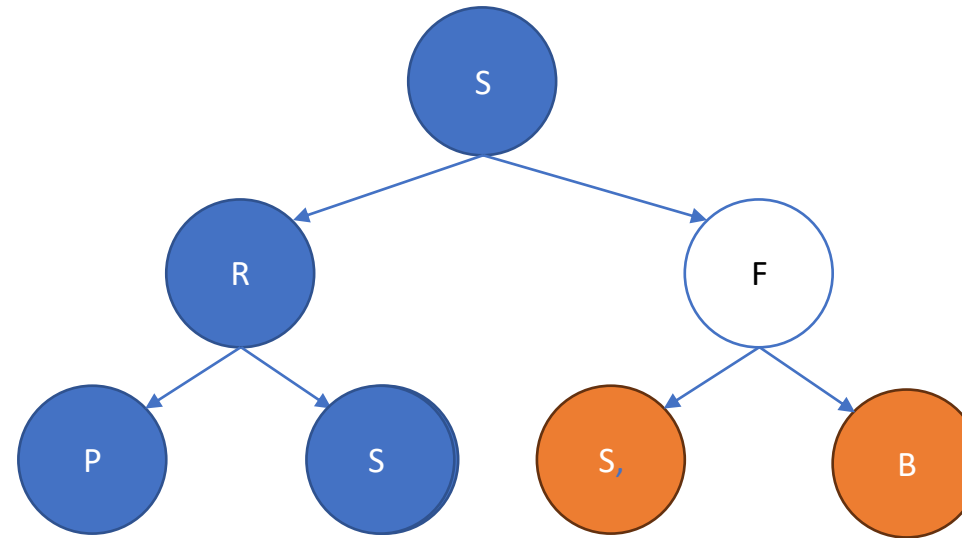
```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```

Queue:



Breadth-first Search (BFS)

```
create frontier : queue
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

```
    if node.state is goal: return solution
```

```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

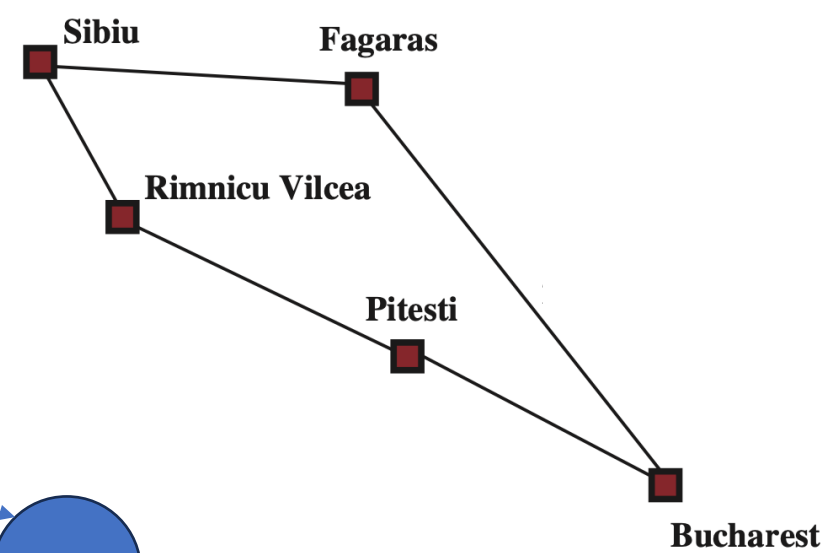
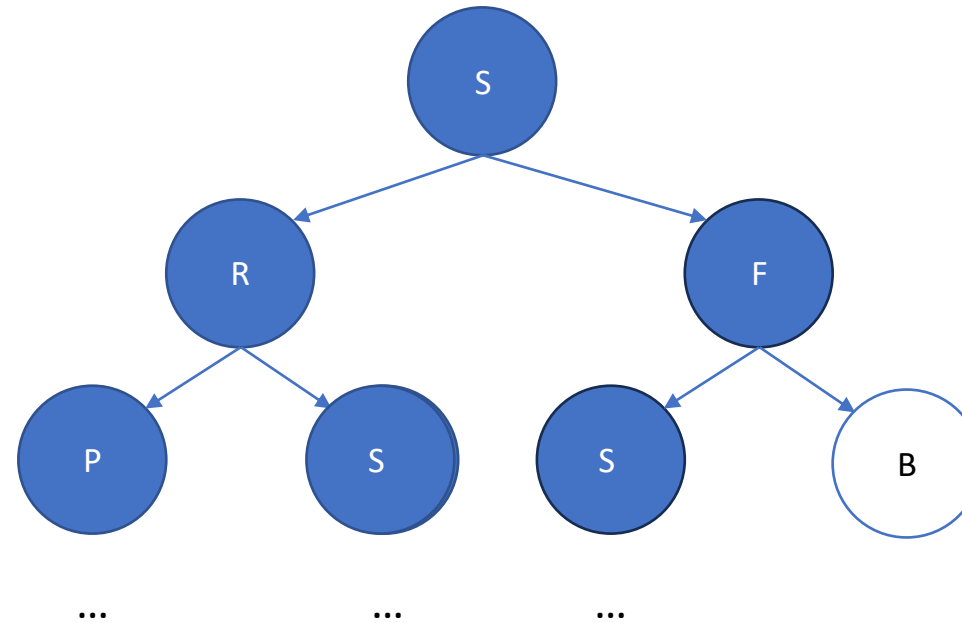
```
        frontier.add(Node(next_state))
```

```
return failure
```

Queue:

B

...



Breadth-first Search (BFS)

```
create frontier : queue
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

```
    if node.state is goal: return solution
```

```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

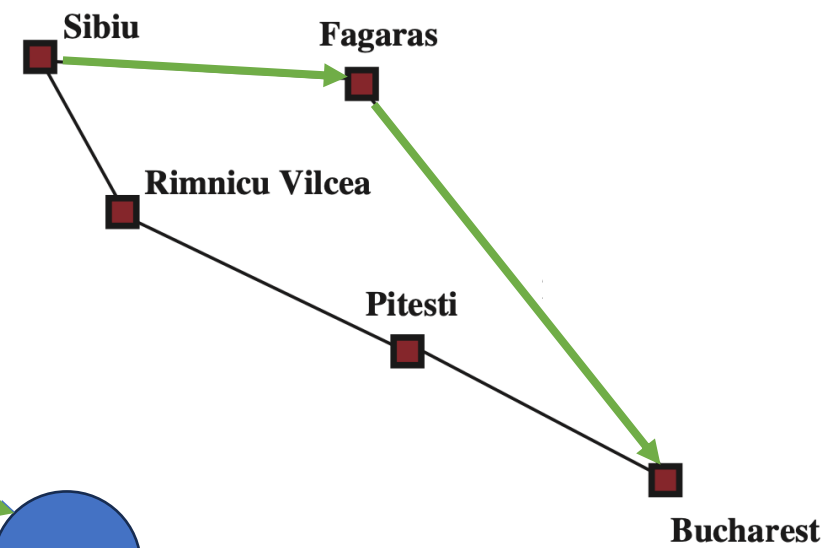
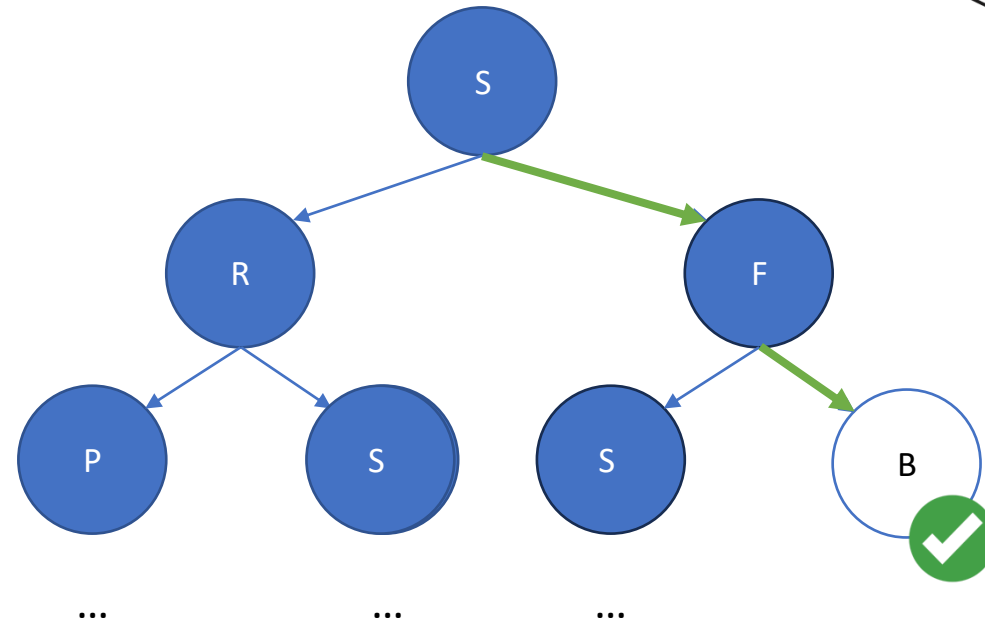
```
        frontier.add(Node(next_state))
```

```
return failure
```

Queue:

B

...



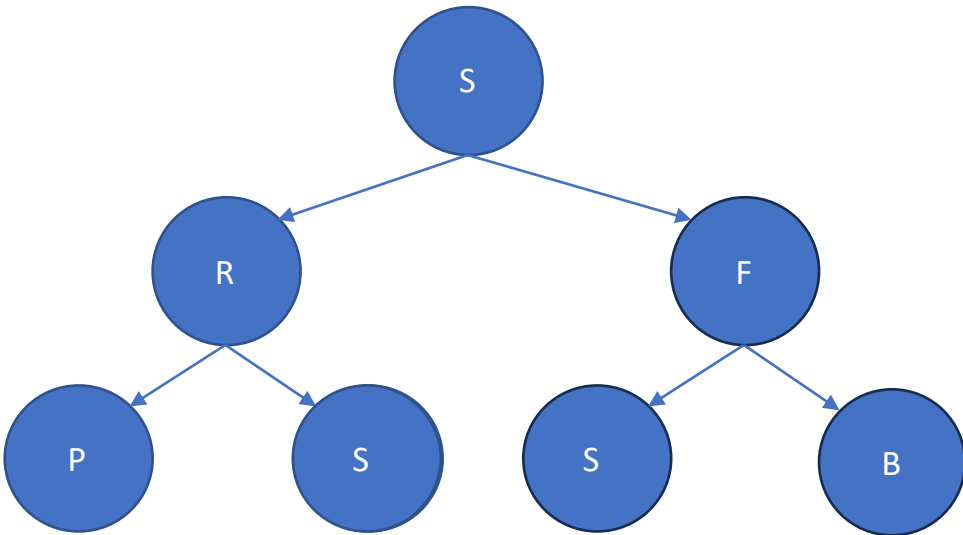
Breadth-first Search (BFS) - Analysis

Time complexity: (# nodes generated)

Exponential w.r.t. the depth of the optimal solution

Space complexity: (size of frontier)

Exponential w.r.t. the depth of the optimal solution



Theorem (Completeness): If the problem has a finite branching factor and there exist a goal at a finite depth, BFS is complete: it will terminate and return a solution.

Theorem (Optimality): Assume all actions have equal positive cost, so each edge has cost $c = 1$ (unit step cost). If a goal exists at finite depth, BFS returns a solution with minimal path cost (i.e., the fewest steps).

Uniform-cost Search (UCS)

Cost from root to a state

```
create frontier : priority queue (path cost)
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

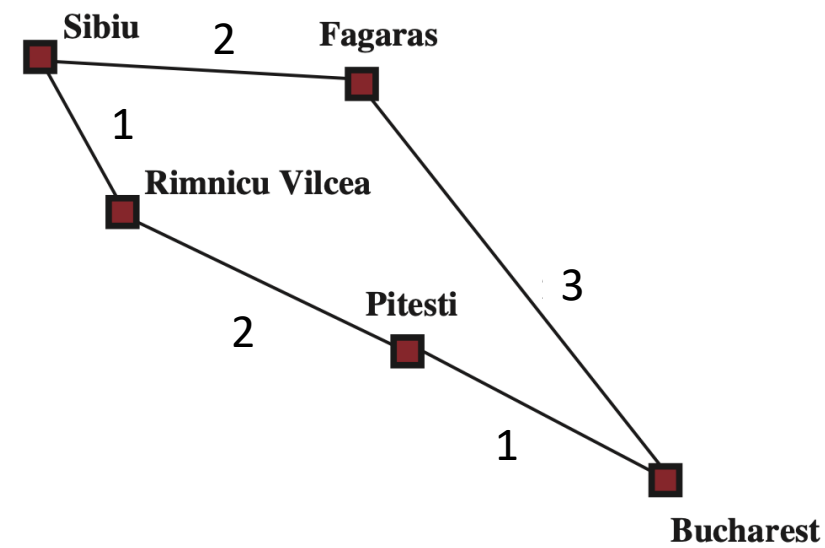
```
    if node.state is goal: return solution
```

```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```



Uniform-cost Search (UCS)

Cost from root to a state

```
create frontier : priority queue (path cost)
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

```
    if node.state is goal: return solution
```

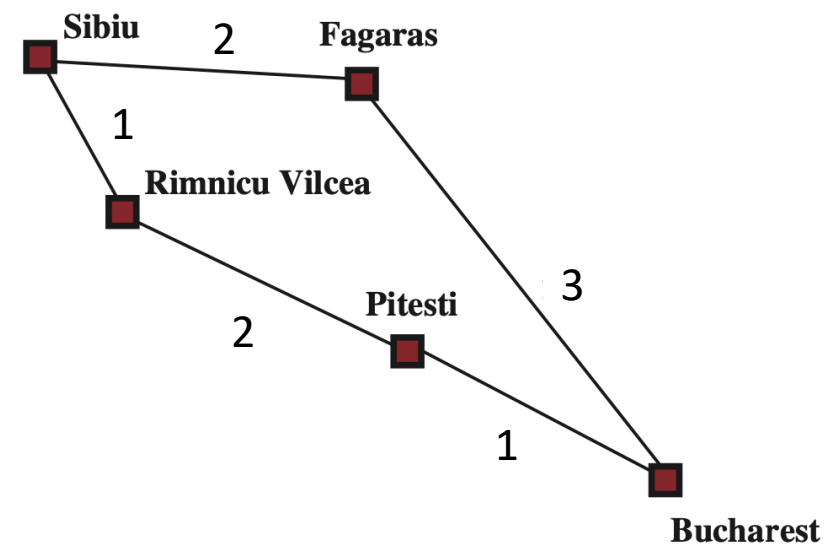
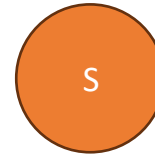
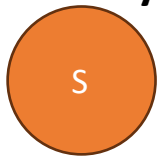
```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```

Priority Queue:



Uniform-cost Search (UCS)

Cost from root to a state

```
create frontier : priority queue (path cost)
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

```
    if node.state is goal: return solution
```

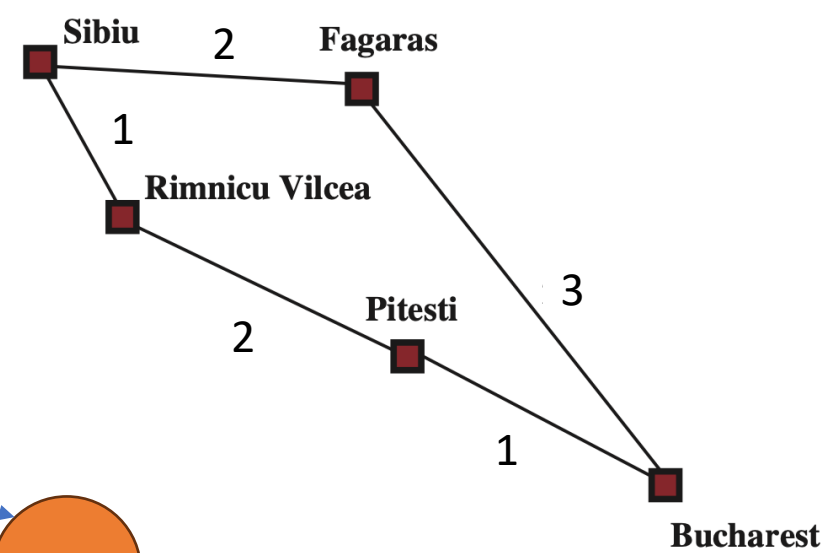
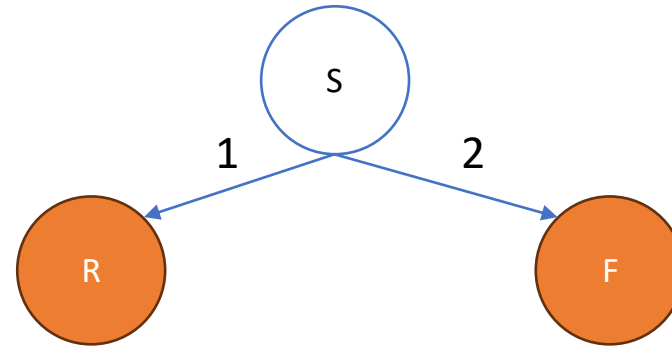
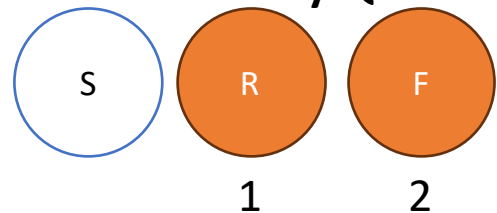
```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```

Priority Queue:



Uniform-cost Search (UCS)

Cost from root to a state

```
create frontier : priority queue (path cost)
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

```
    if node.state is goal: return solution
```

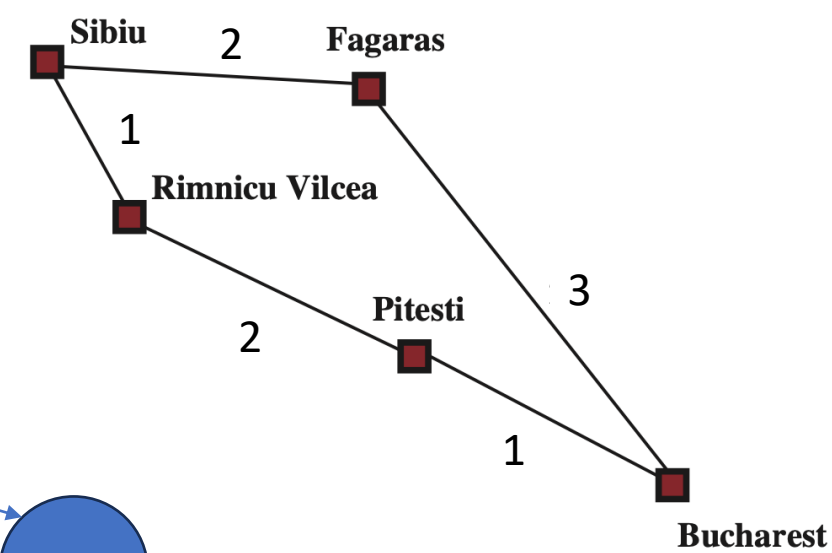
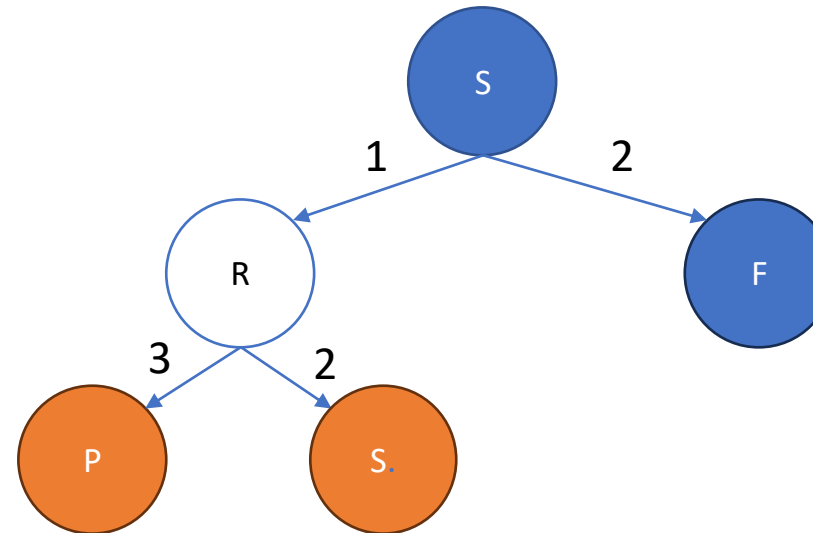
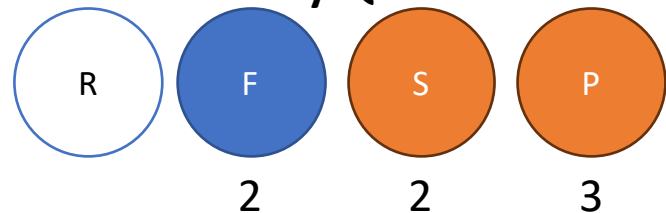
```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```

Priority Queue:



Uniform-cost Search (UCS)

Cost from root to a state

```
create frontier : priority queue (path cost)
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

```
    if node.state is goal: return solution
```

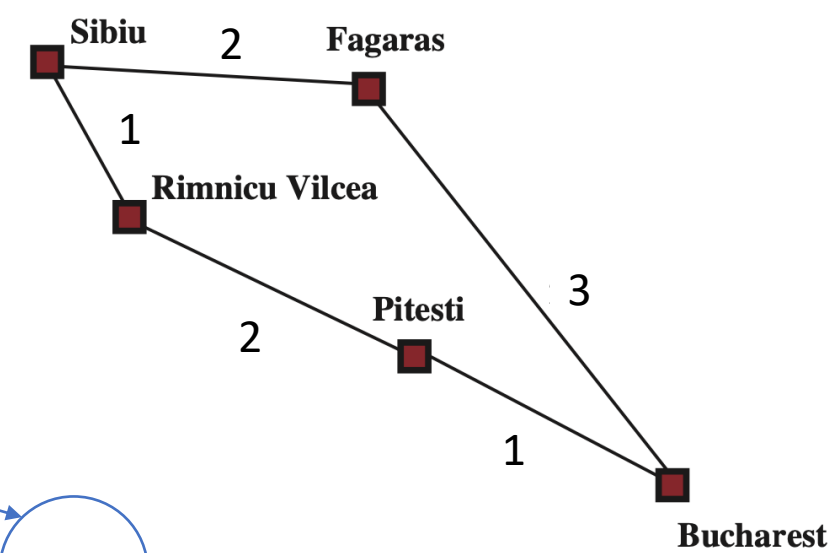
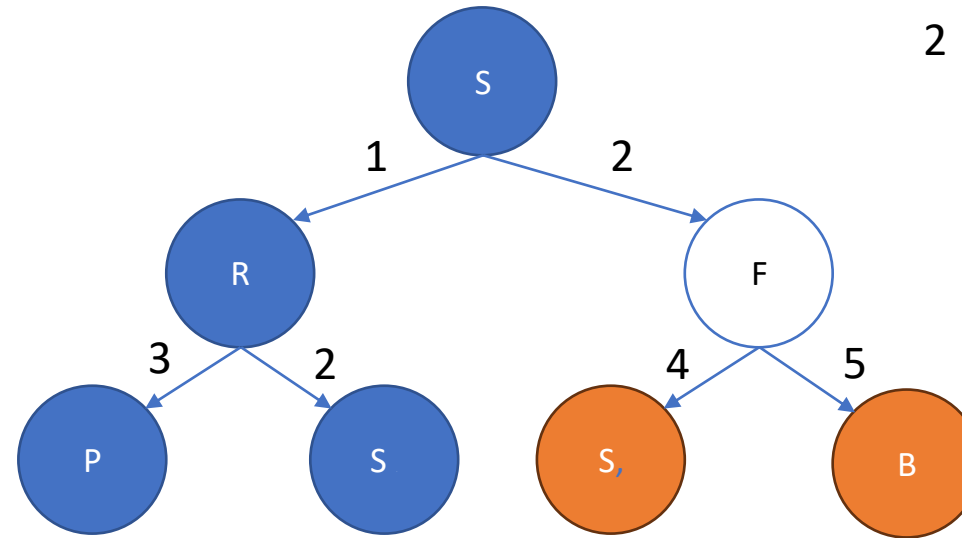
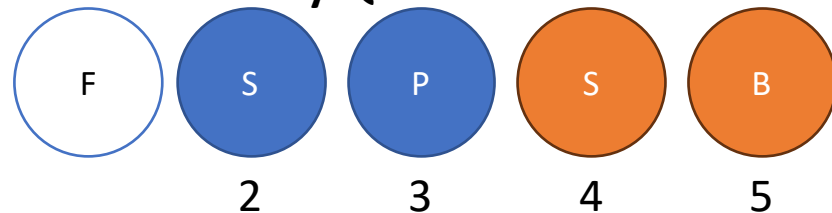
```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```

Priority Queue:



Uniform-cost Search (UCS)

Cost from root to a state

create **frontier** : **priority queue** (path cost)

insert **Node**(initial_state) to **frontier**

while **frontier** is not empty:

node = **frontier**.pop()

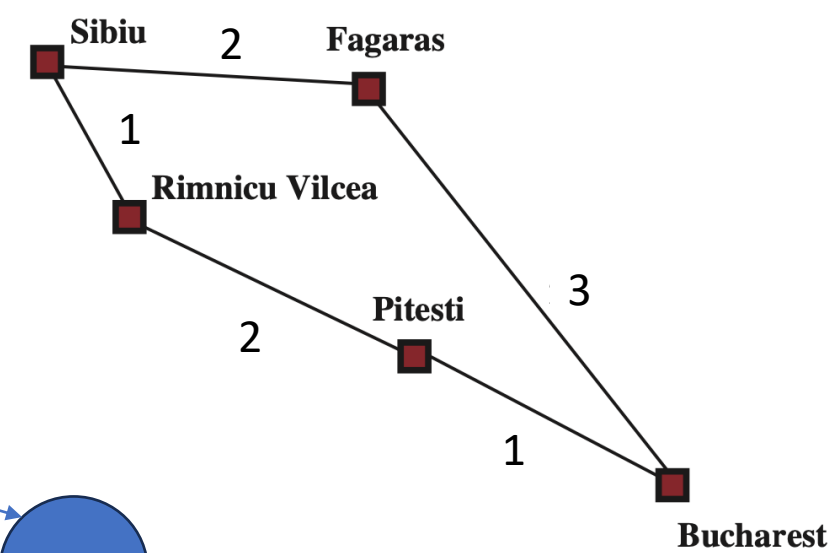
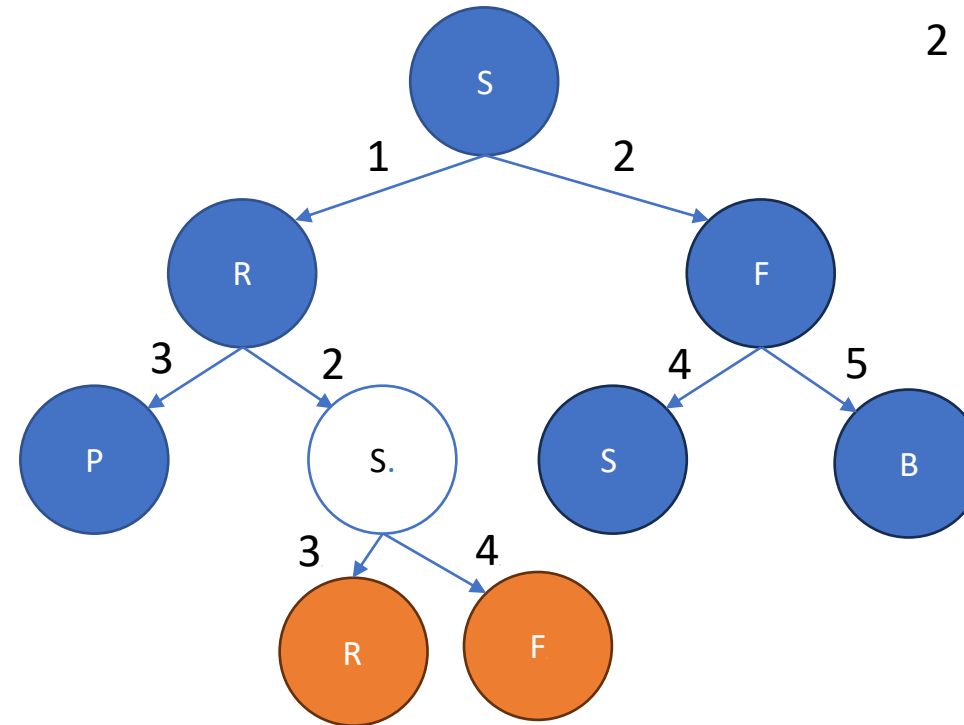
 if **node.state** is goal: return solution

 for **action** in actions(**node.state**):

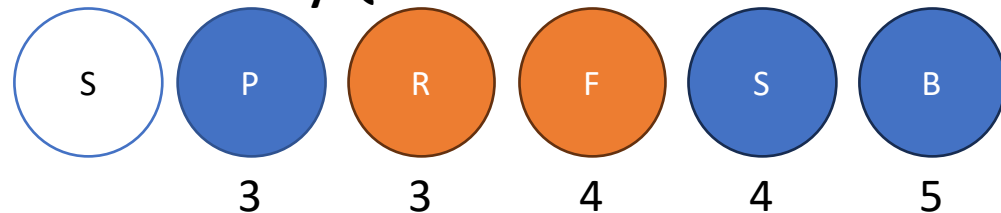
next_state = transition(**node.state**, **action**)

frontier.add(**Node**(**next_state**))

return failure



Priority Queue:



Uniform-cost Search (UCS)

Cost from root to a state

```
create frontier : priority queue (path cost)
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

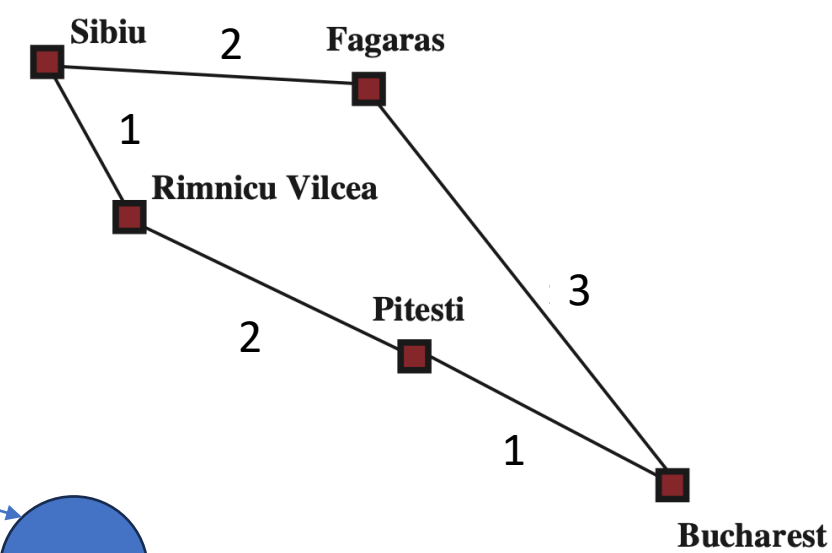
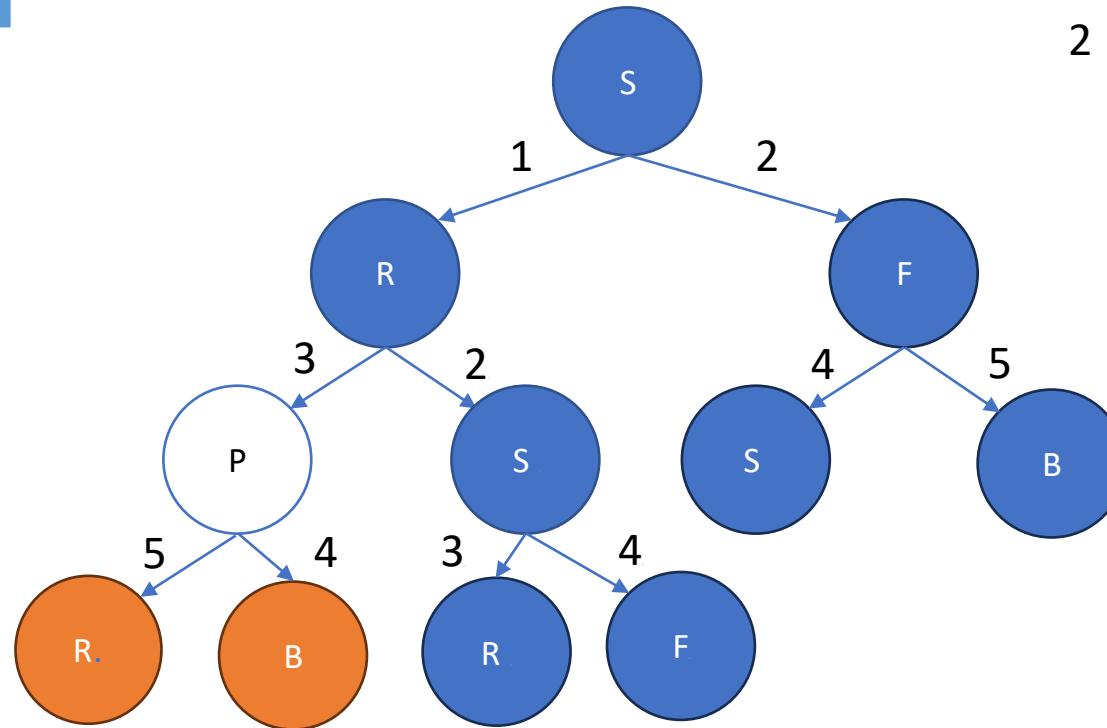
```
    if node.state is goal: return solution
```

```
    for action in actions(node.state):
```

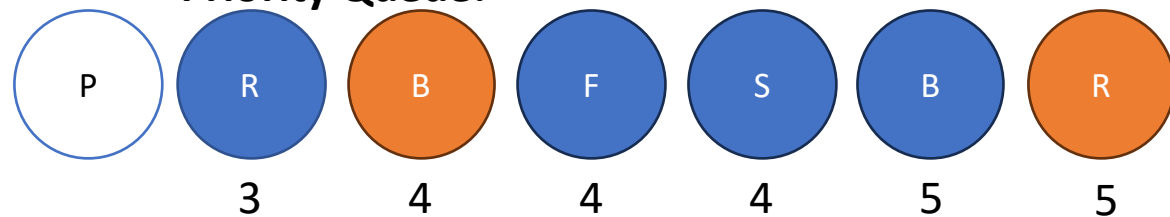
```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```



Priority Queue:



Uniform-cost Search (UCS)

Cost from root to a state

```
create frontier : priority queue (path cost)
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

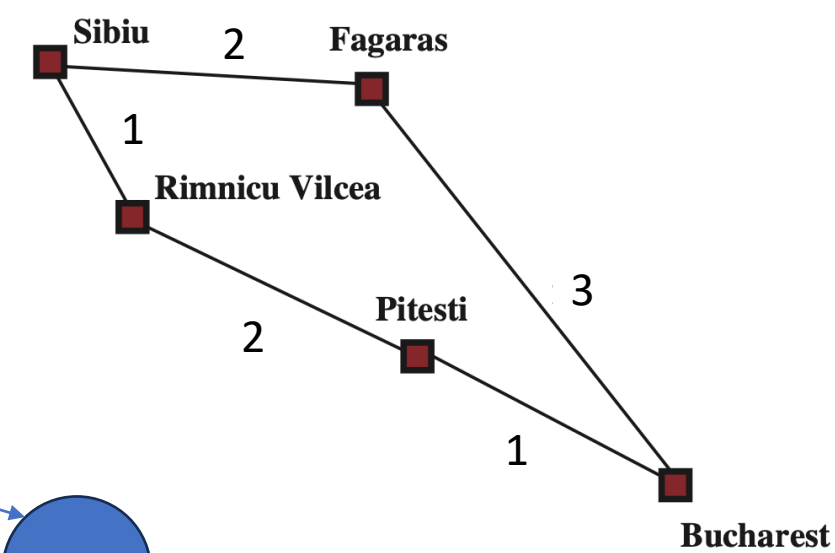
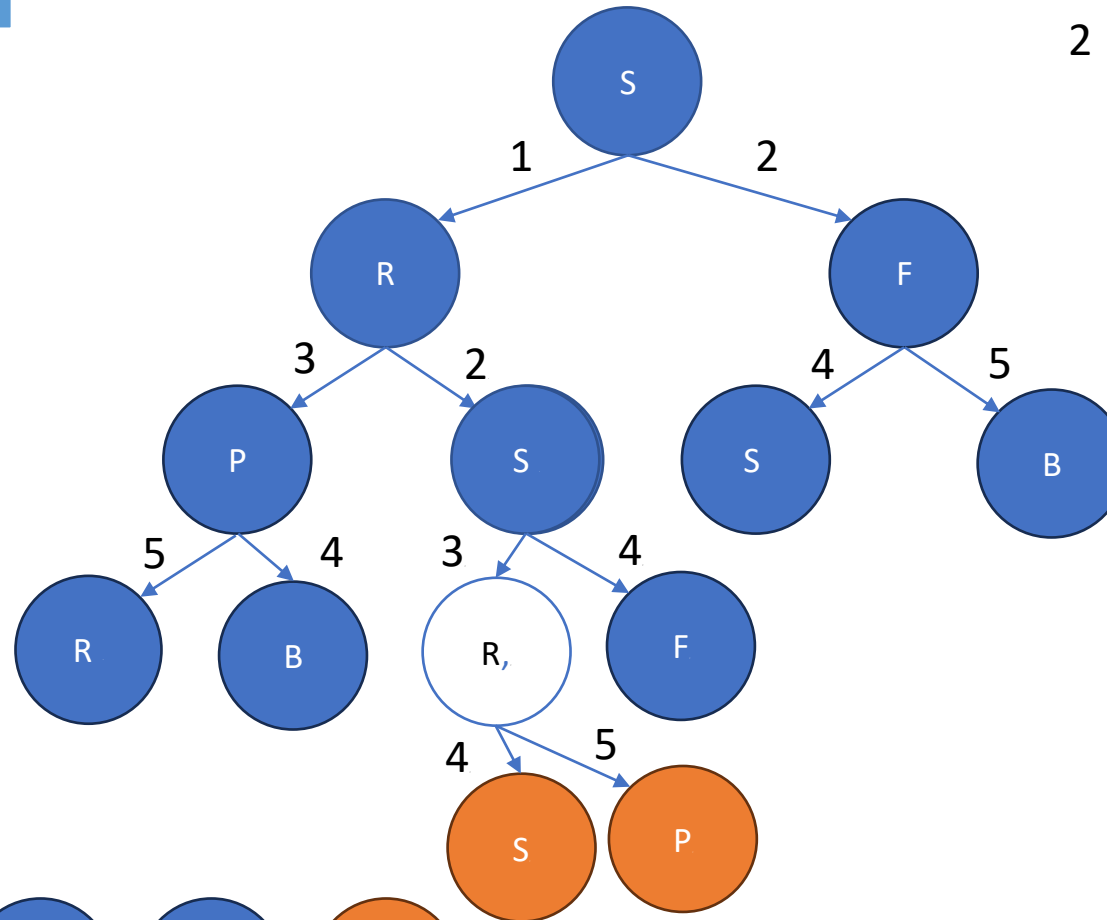
```
    if node.state is goal: return solution
```

```
    for action in actions(node.state):
```

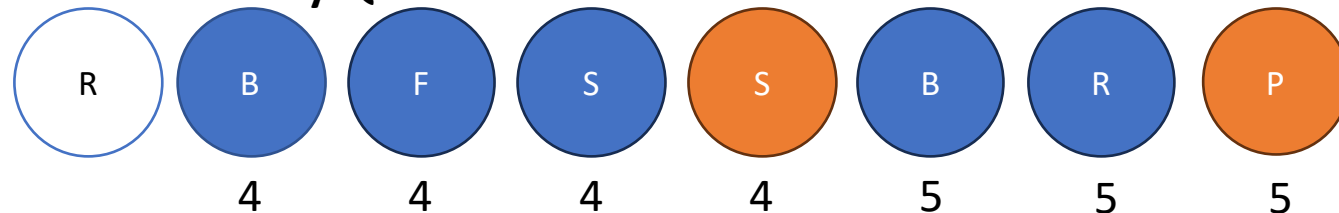
```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```



Priority Queue:



Uniform-cost Search (UCS)

Cost from root to a state

create **frontier** : **priority queue** (path cost)

insert **Node**(initial_state) to **frontier**

while **frontier** is not empty:

node = **frontier.pop**()

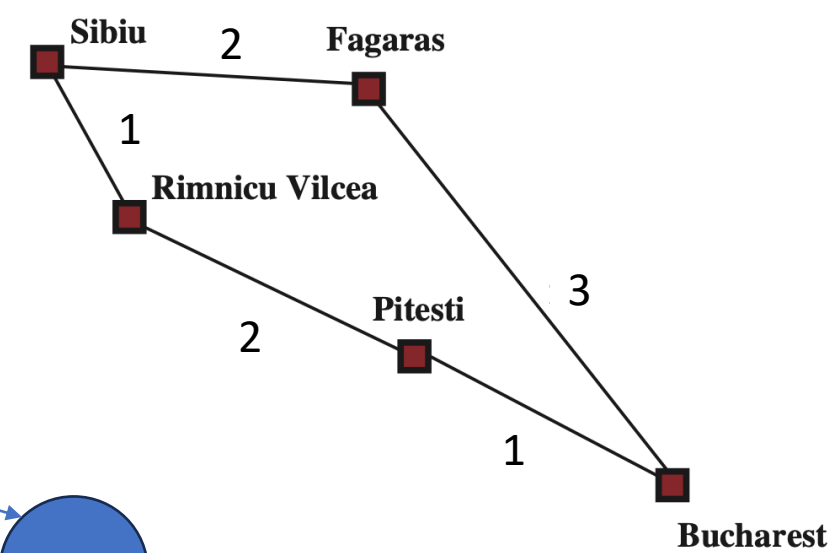
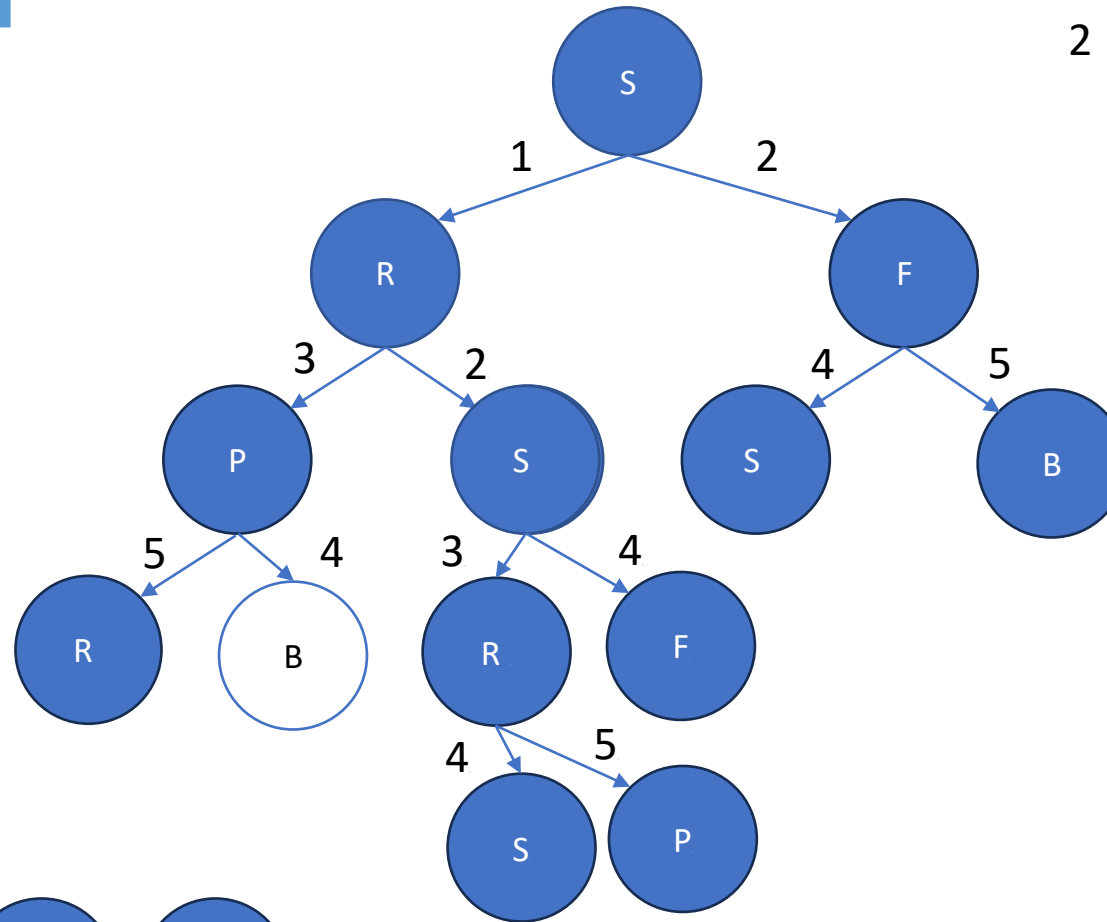
if **node.state** is goal: return solution

for **action** in **actions**(**node.state**):

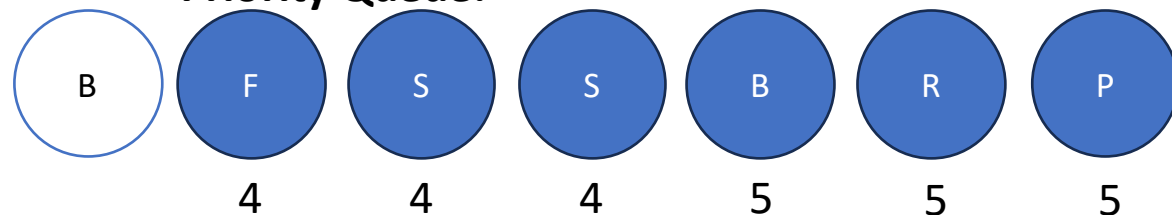
next_state = **transition**(**node.state**, **action**)

frontier.add(**Node**(**next_state**))

return failure



Priority Queue:



Uniform-cost Search (UCS)

Cost from root to a state

create **frontier** : **priority queue** (path cost)

insert **Node**(initial_state) to **frontier**

while **frontier** is not empty:

node = **frontier**.pop()

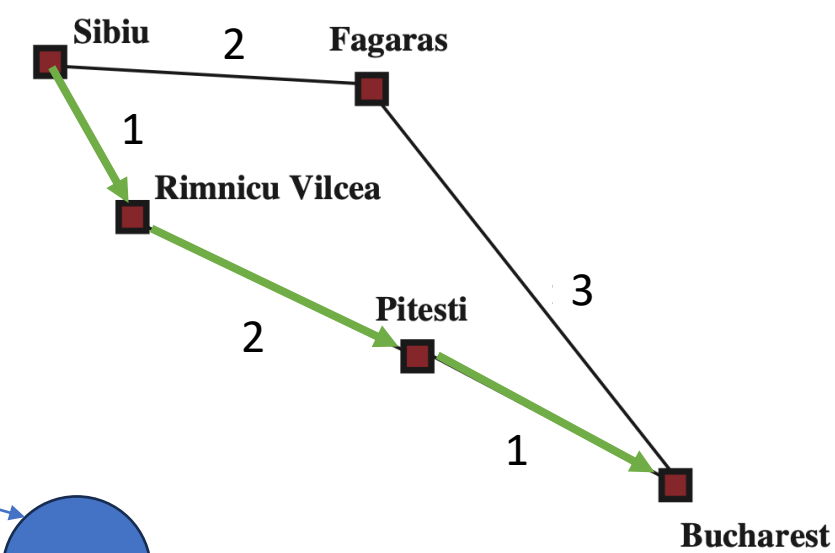
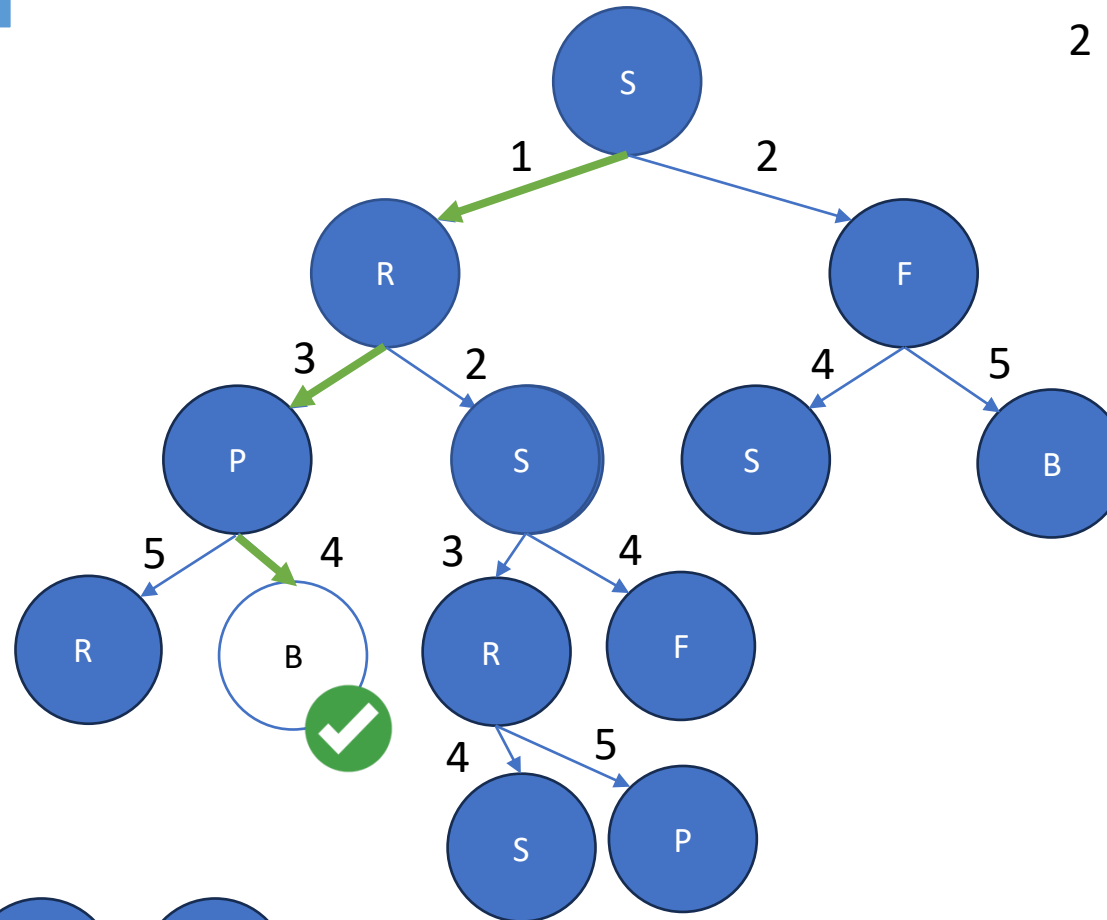
if **node.state** is goal: return solution

for **action** in actions(**node.state**):

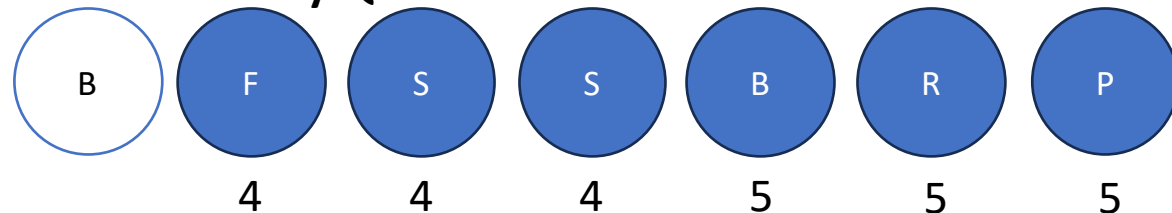
next_state = transition(**node.state**, **action**)

frontier.add(**Node**(**next_state**))

return failure



Priority Queue:



Uniform-cost Search (UCS)

Cost from root to a state

create **frontier** : priority queue (path cost)

insert **Node**(initial_state) to **frontier**

while **frontier** is not empty:

node = **frontier**.pop()

 if **node.state** is goal: return solution

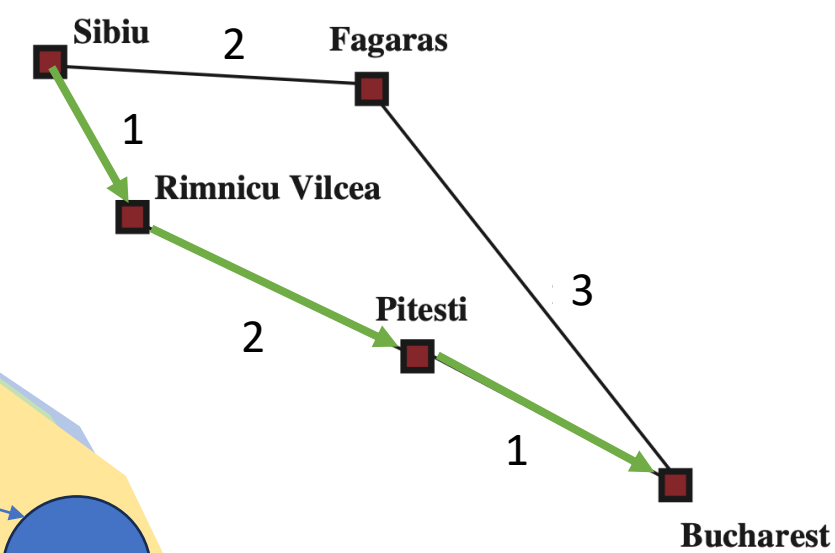
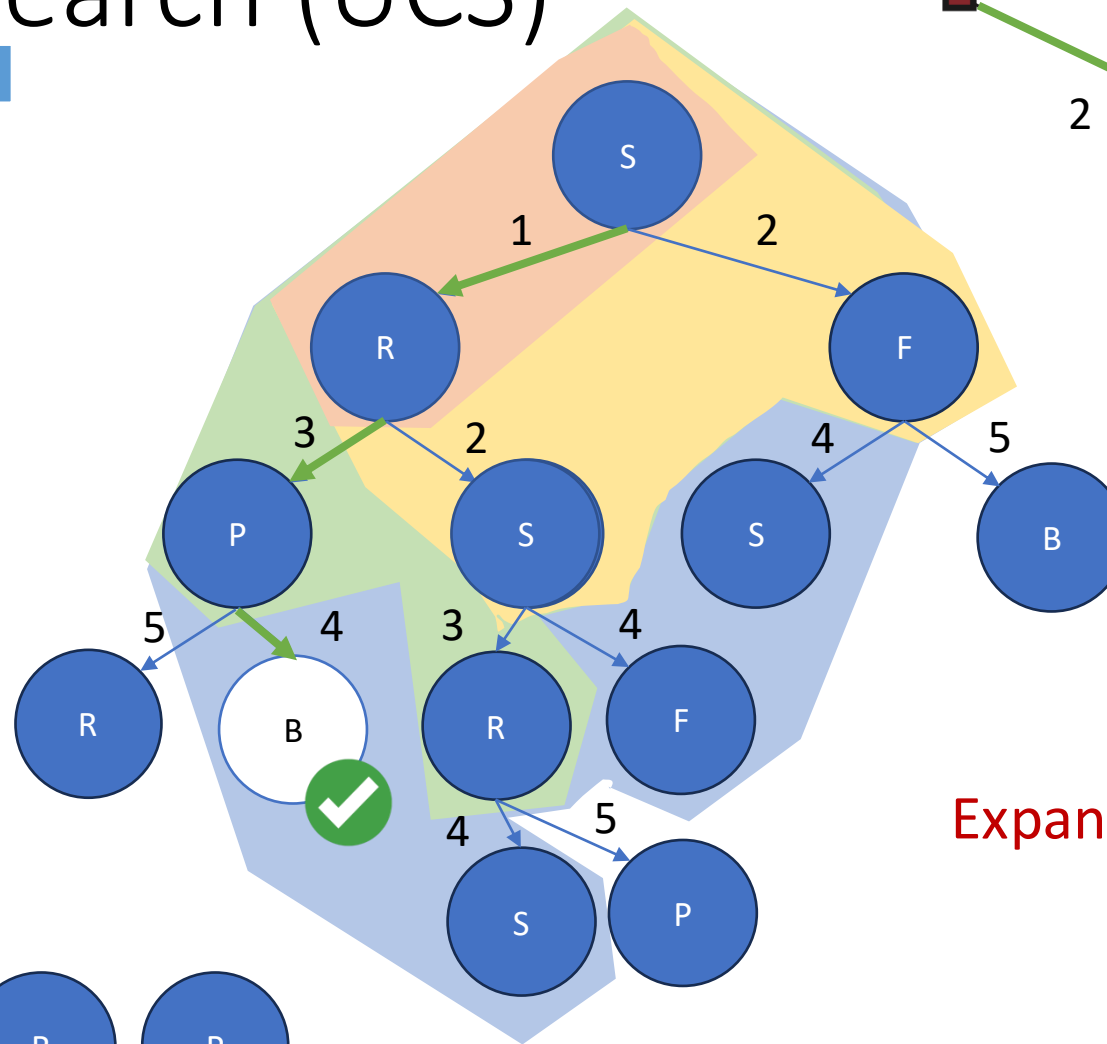
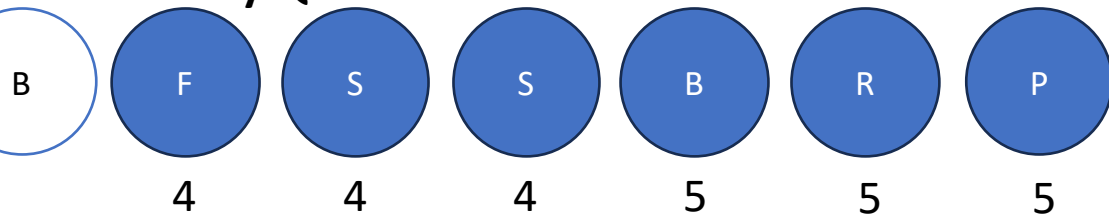
 for **action** in actions(**node.state**):

next_state = transition(**node.state**, **action**)

frontier.add(**Node**(**next_state**))

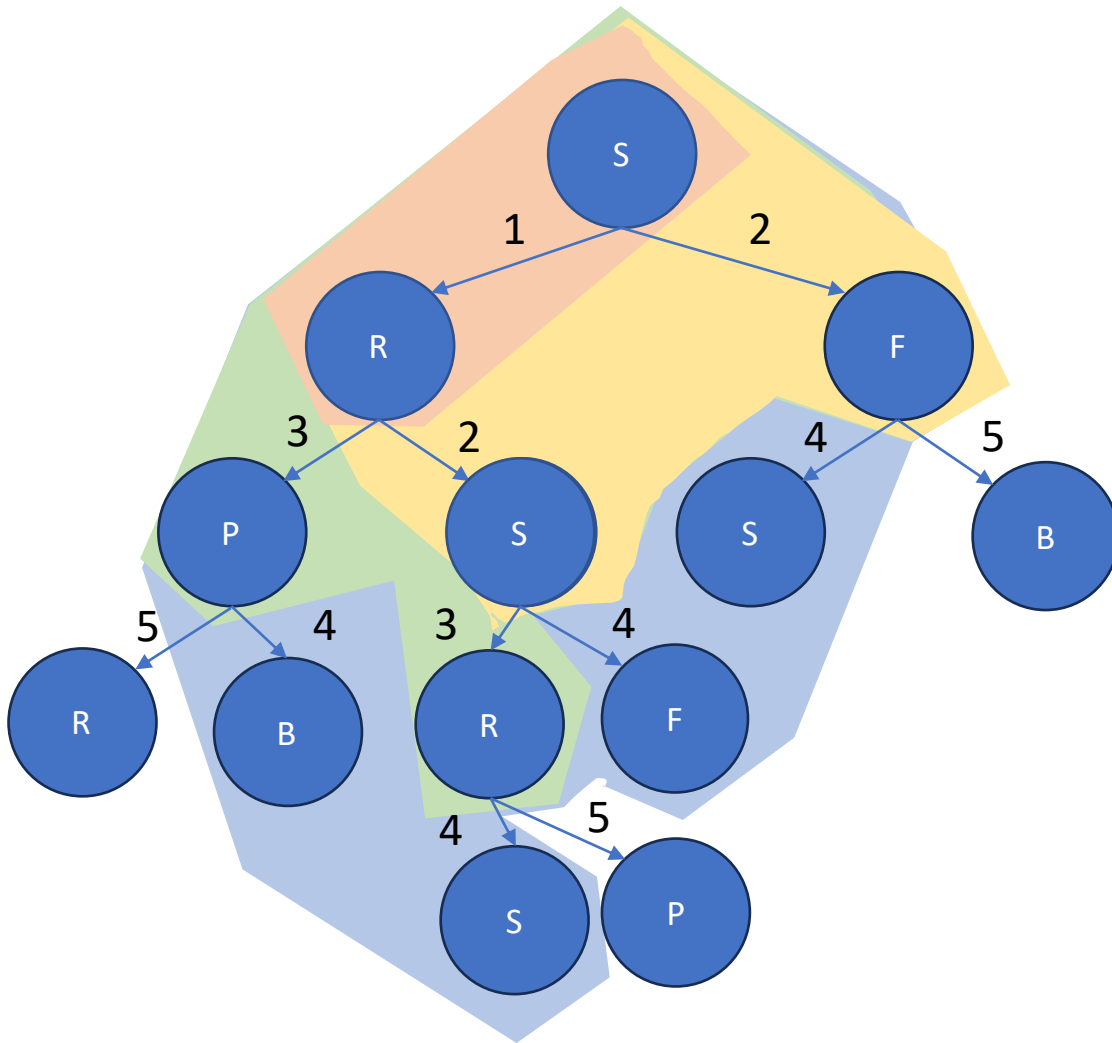
return failure

Priority Queue:



Expands “tier” by “tier”

Uniform-cost Search (UCS) - Analysis



Time complexity: (# nodes generated)

Exponential w.r.t. the “tier” of the optimal solution

Space complexity: (size of frontier)

Exponential w.r.t. the “tier” of the optimal solution

Theorem (Completeness): Let the search tree have finite branching factor and let every action cost satisfy $c(n, a, n') \geq \epsilon > 0$, for all node n , action a , and next node n' . If a goal is reachable with finite optimal path cost $C^* < \infty$, then UCS is complete: it will find a goal.

Theorem (Optimality): If all action costs are nonnegative, $c(n, a, n') \geq 0$, then UCS is optimal: the first goal node it removes from the priority queue has minimal total path cost C^* .

Notes:

- The positive lower bound $\epsilon > 0$ prevents infinitely many distinct nodes with total cost below C^* , ensuring completeness.
- UCS remains optimal with zero-cost edges.

Depth-first Search (DFS)

```
create frontier : stack
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

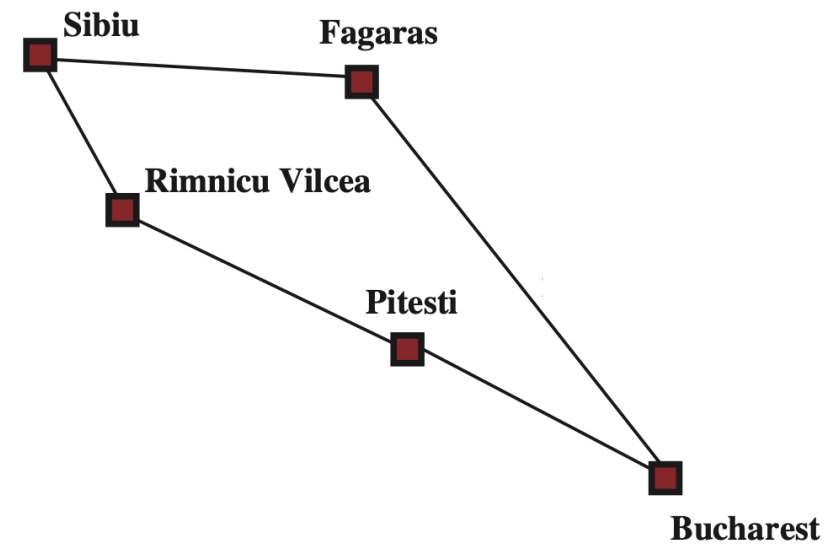
```
    if node.state is goal: return solution
```

```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```



Depth-first Search (DFS)

```
create frontier : stack
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

```
    if node.state is goal: return solution
```

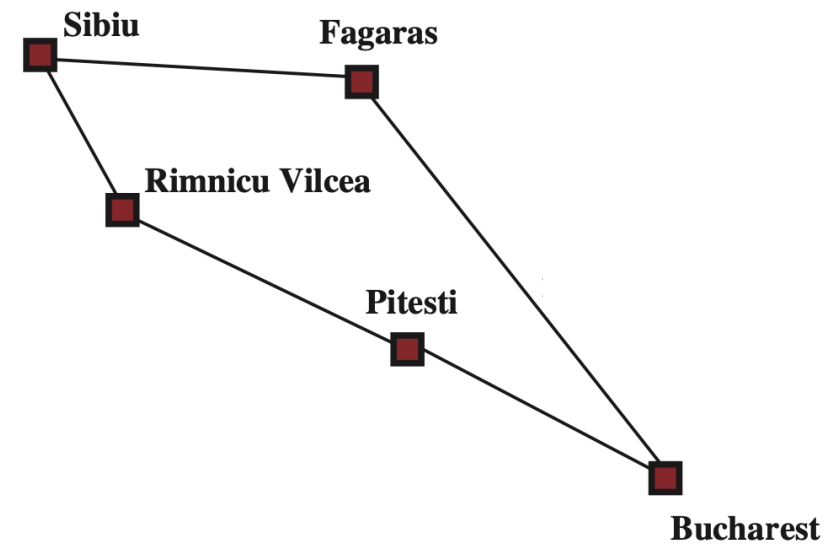
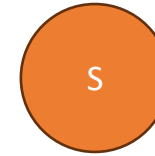
```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```

Stack:



Depth-first Search (DFS)

```
create frontier : stack
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

```
    if node.state is goal: return solution
```

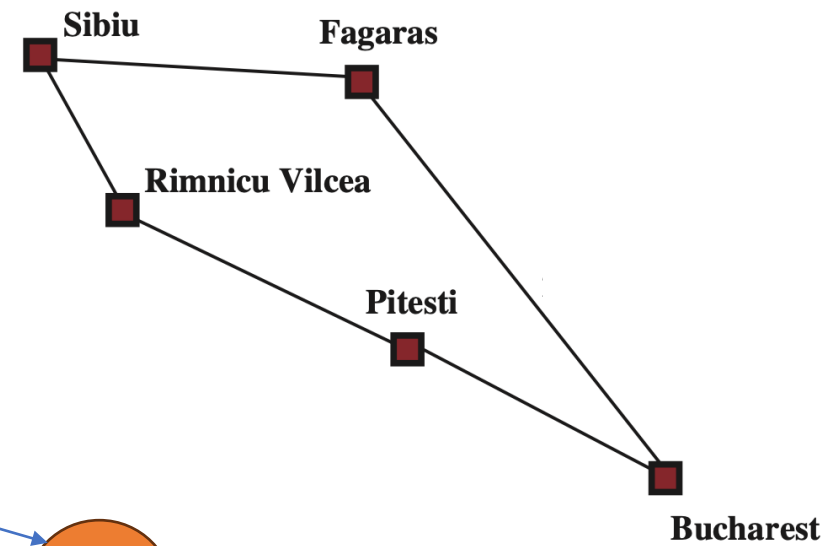
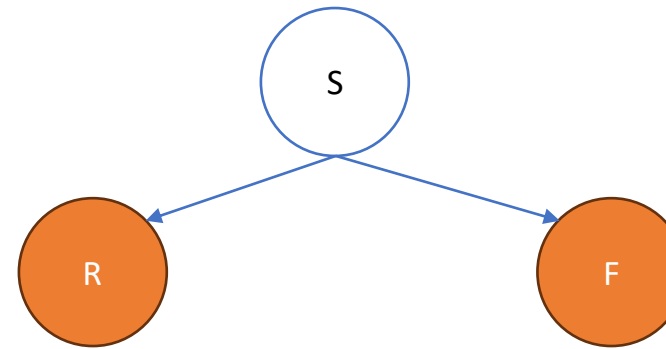
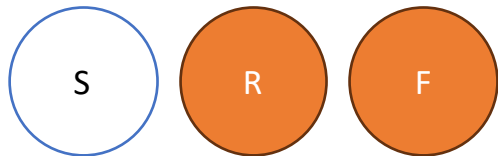
```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```

Stack:



Depth-first Search (DFS)

```
create frontier : stack
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

```
    if node.state is goal: return solution
```

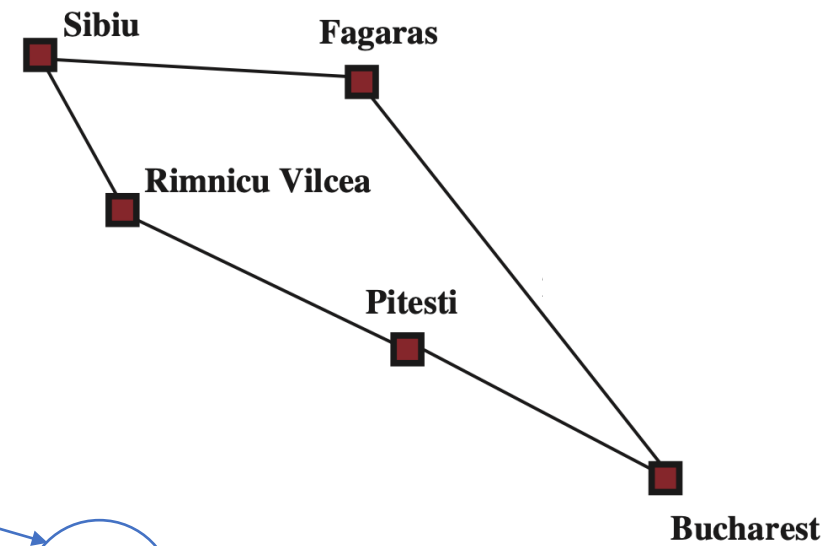
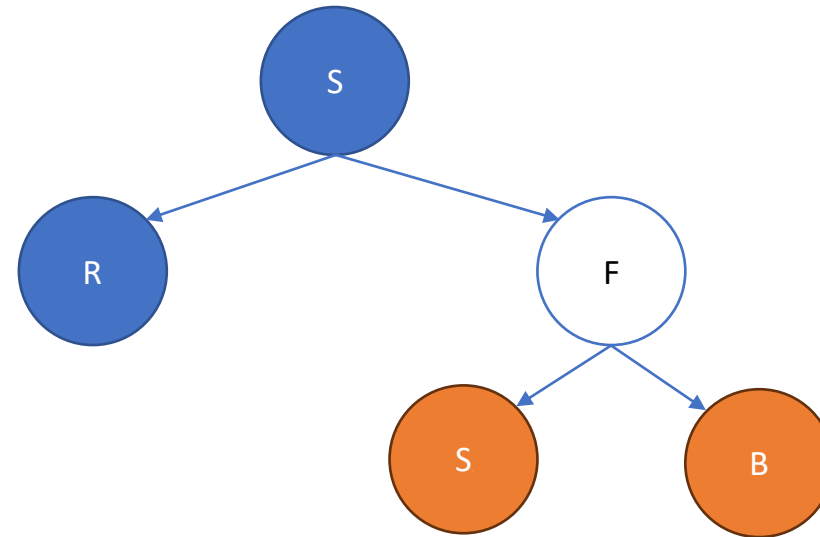
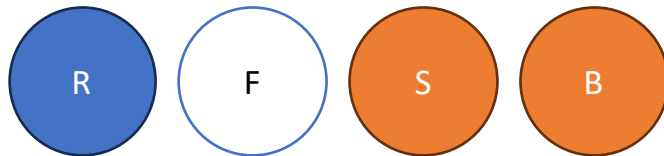
```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

```
return failure
```

Stack:



Depth-first Search (DFS)

create **frontier** : **stack**

insert **Node**(initial_state) to **frontier**

while **frontier** is not empty:

node = **frontier**.pop()

 if **node.state** is goal: return solution

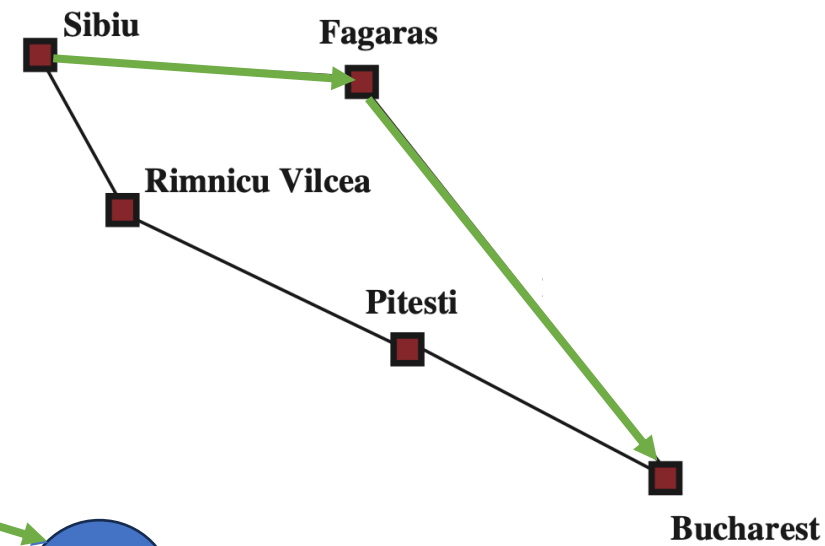
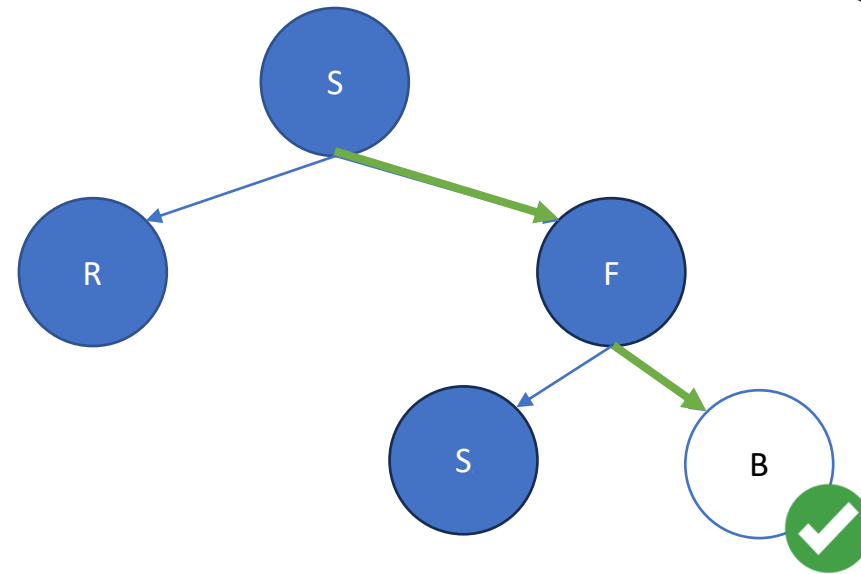
 for **action** in actions(**node.state**):

next_state = transition(**node.state**, **action**)

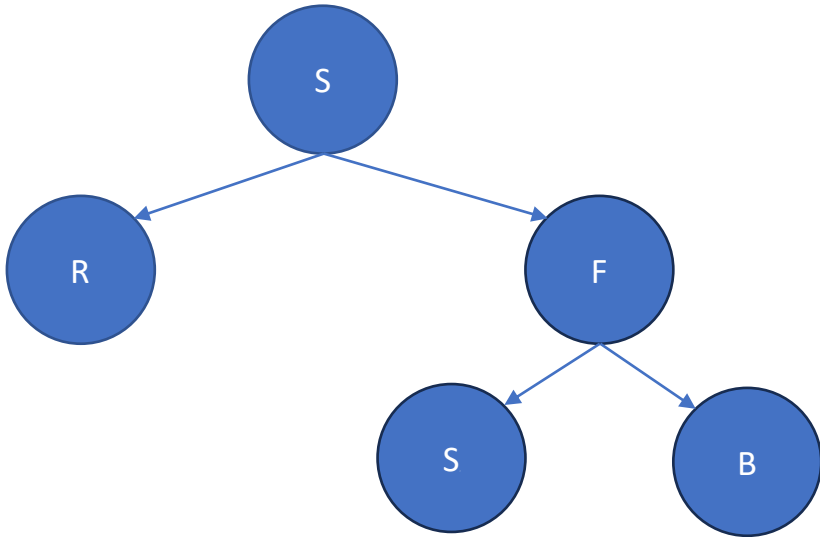
frontier.add(**Node**(**next_state**))

return failure

Stack:



Depth-first Search (DFS) - Analysis



Time complexity: (# nodes generated)

Exponential w.r.t. the maximum depth of the search tree

Space complexity: (size of frontier)

Polynomial w.r.t. the maximum depth of the search tree

Complete: **No**, when the depth of the search tree is infinite (e.g., when the action is reversible, causing the state to go back and forth)

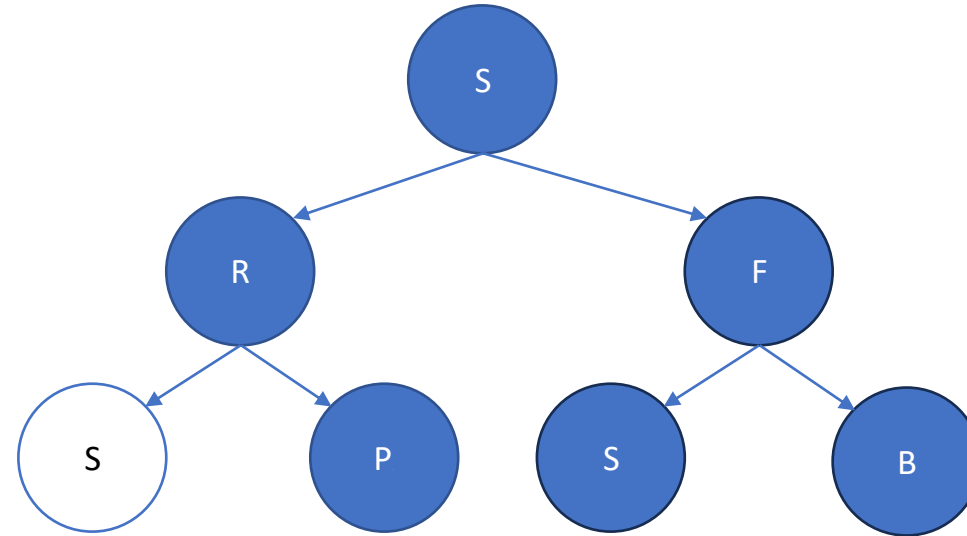
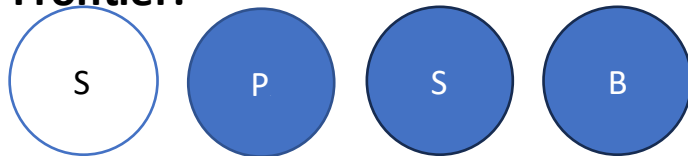
Optimal: **No**, the optimal solution may be in a shallower depth

Search with visited memory

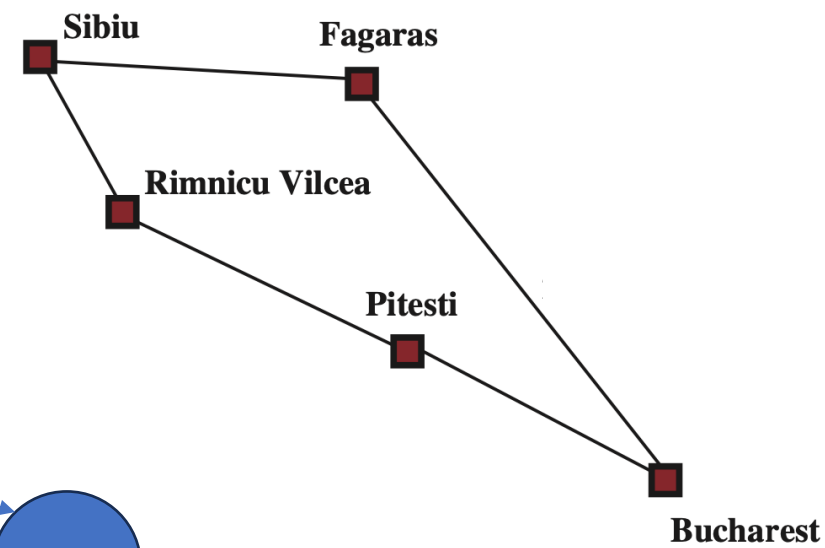
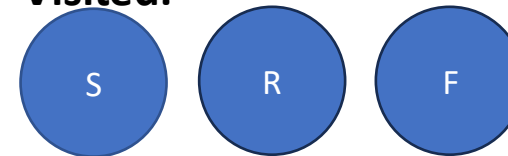
AIAMA: graph search

```
create frontier
create visited
insert Node(initial_state) to frontier
while frontier is not empty:
    node = frontier.pop()
    if node.state is goal: return solution
    if node.state in visited: continue
    visited.add(state)
    for action in actions(node.state):
        next_state = transition(node.state, action)
        frontier.add(Node(next_state))
return failure
```

Frontier:



Visited:



Poll Everywhere

PollEV.com/rizki

Grid World problem:

- Agent is in the middle,
- Agent can move left or right,
- Agent cannot move to a place that it has visited before.



Which search algorithm(s) without visited memory is/are **optimal** and the most **time** and **space** efficient (we care about constants)?

- Breadth-first Search
- Uniform-cost Search
- Depth-first Search

Poll Everywhere

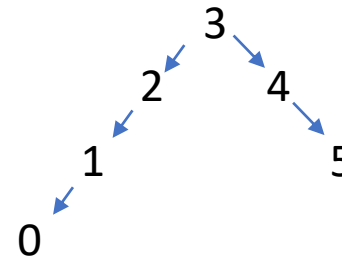
Grid World problem:

- Agent is in the middle,
- Agent can move left or right,
- Agent cannot move to a place that it has visited before.



Which search algorithm(s) without visited memory is/are **optimal** and the most **time** and **space** efficient (we care about constants)?

- Breadth-first Search
- Uniform-cost Search
- Depth-first Search (correct)



BFS/UCS



Break

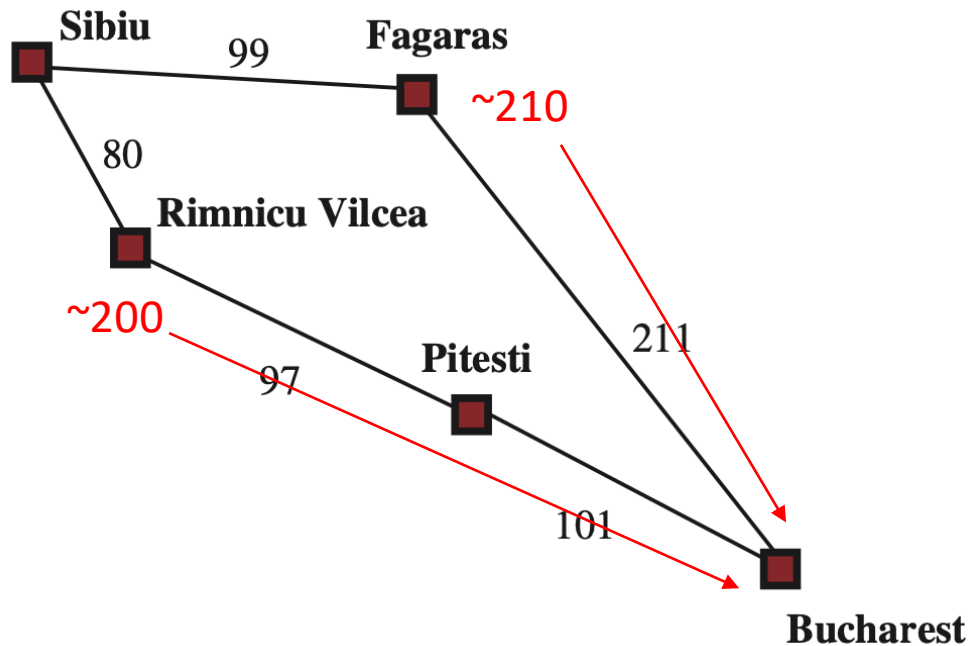
CAN YOU
SOLVE THE
RIVER CROSSING RIDDLE?



Outline

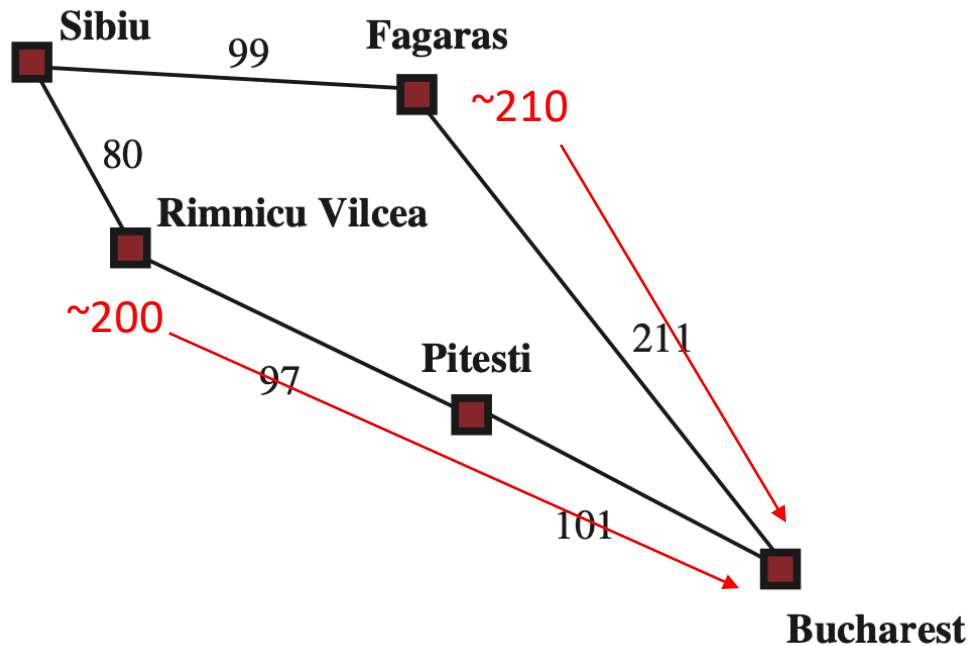
- Designing an agent for search problems
- Search algorithms primer
- Uninformed search
 - Breadth-first, depth-first, and uniform-cost search
- **Informed search**
 - A* search
 - Heuristics
- Search strategies
 - Depth-limited and iterative-deepening search

Informed Search Algorithms



- The search algorithms use **domain-specific hints** about the location of goal(s).
- E.g., an informed agent trying to reach Bucharest will know whether going to Fagaras or Rimnicu Vilcea is a better step (at least approximately).
- Hints = heuristic function.

Heuristics



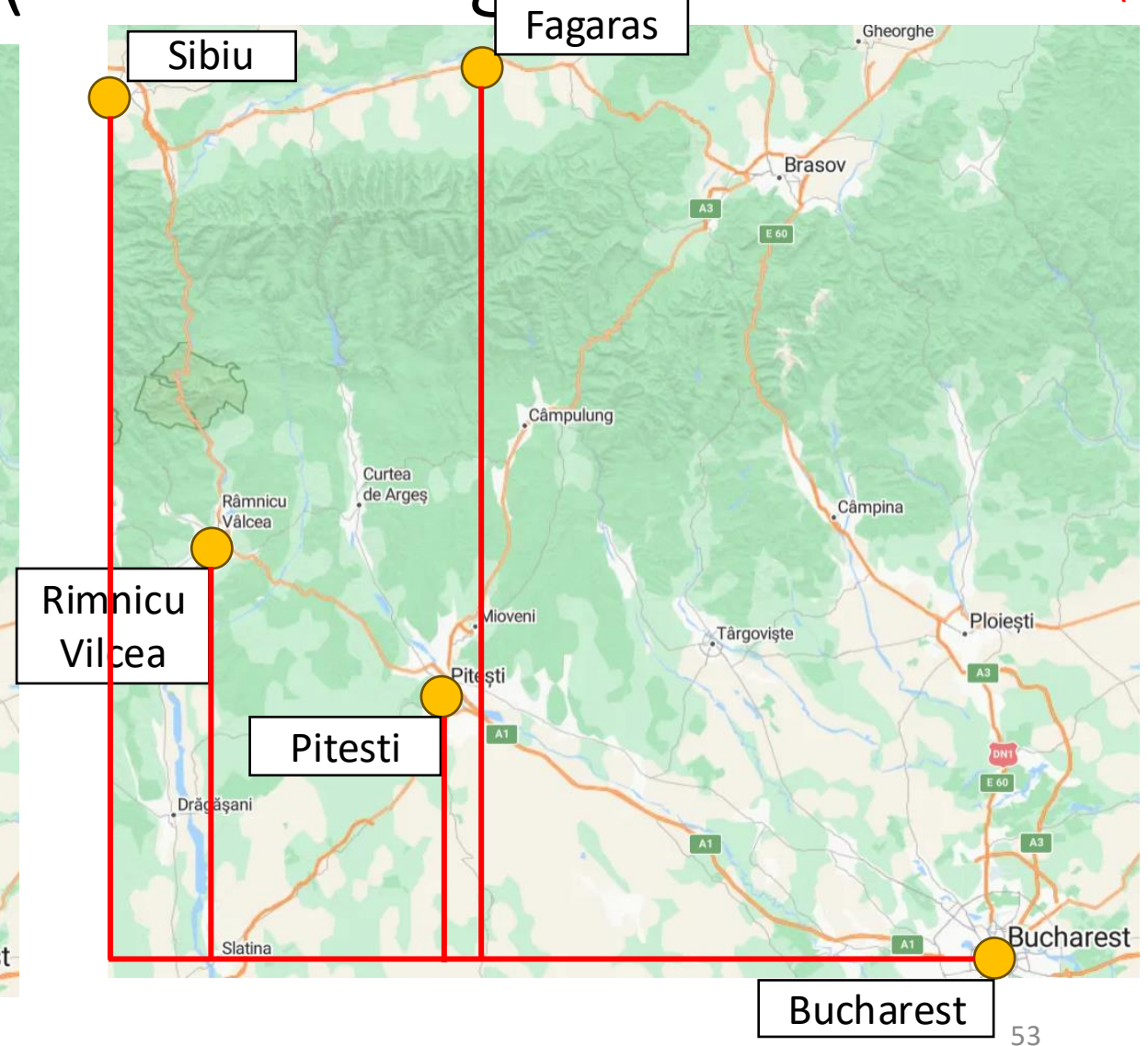
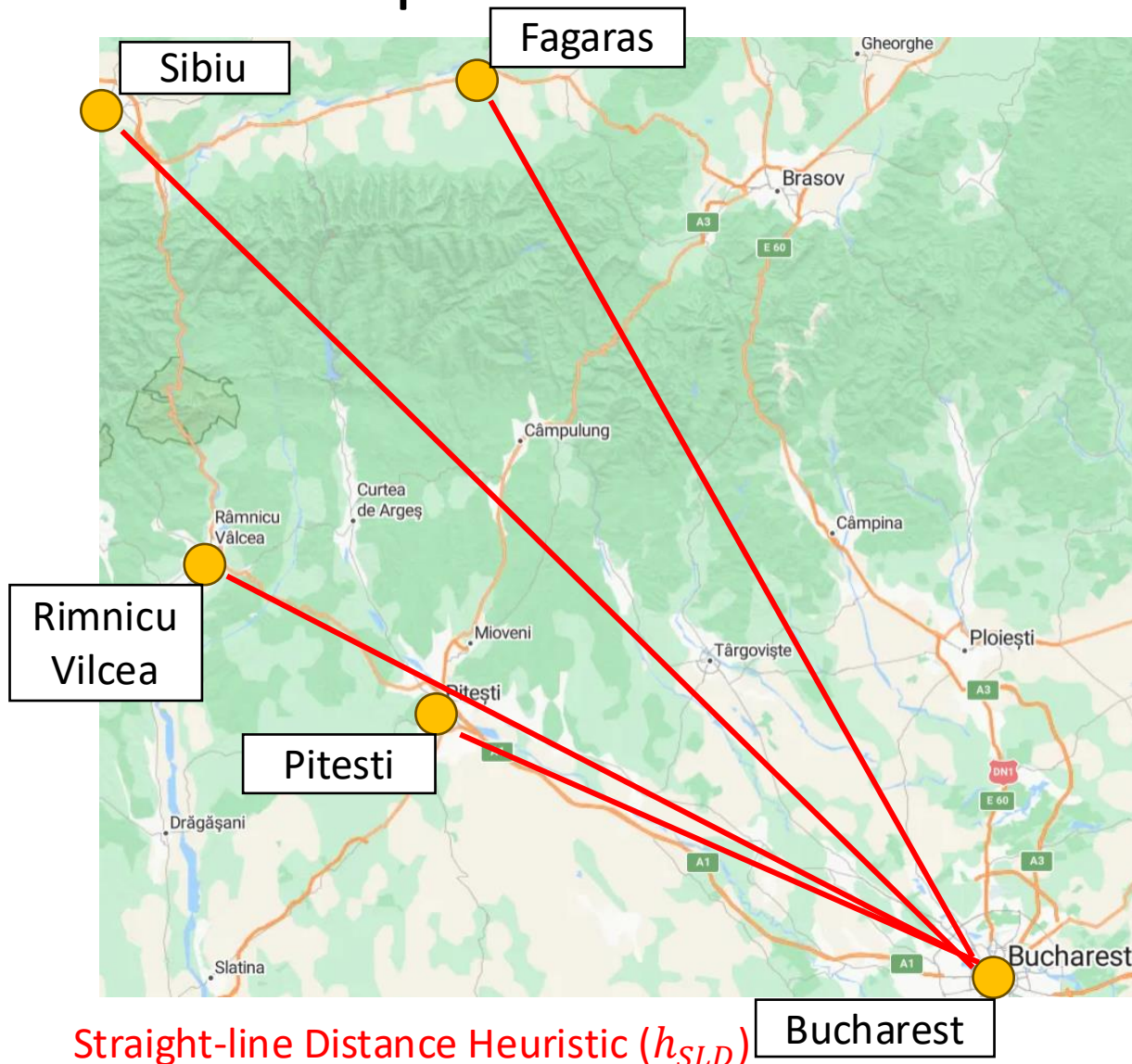
Recall that:

- A sequence of actions forms a **path**.
- An **optimal path** has the lowest path cost among all paths.
- An **optimal path cost** is the cost of optimal path.

A **heuristic** $h(n)$ is an estimate of the optimal path cost from the state at node n to a goal state. A heuristic must be nonnegative and $h(goal) = 0$.

Example of Heuristics (Pathfinding)

Manhattan Distance Heuristic (h_{MD})



Greedy Best-first Search

Evaluation function

```
create frontier : priority queue ( $f(n)$ )
```

```
insert Node(initial_state) to frontier
```

```
while frontier is not empty:
```

```
    node = frontier.pop()
```

```
    if node.state is goal: return solution
```

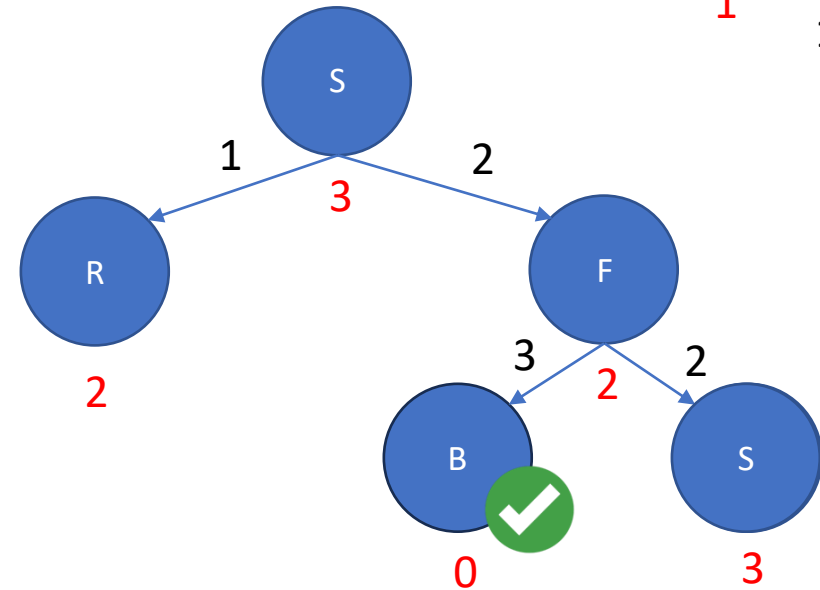
```
    for action in actions(node.state):
```

```
        next_state = transition(node.state, action)
```

```
        frontier.add(Node(next_state))
```

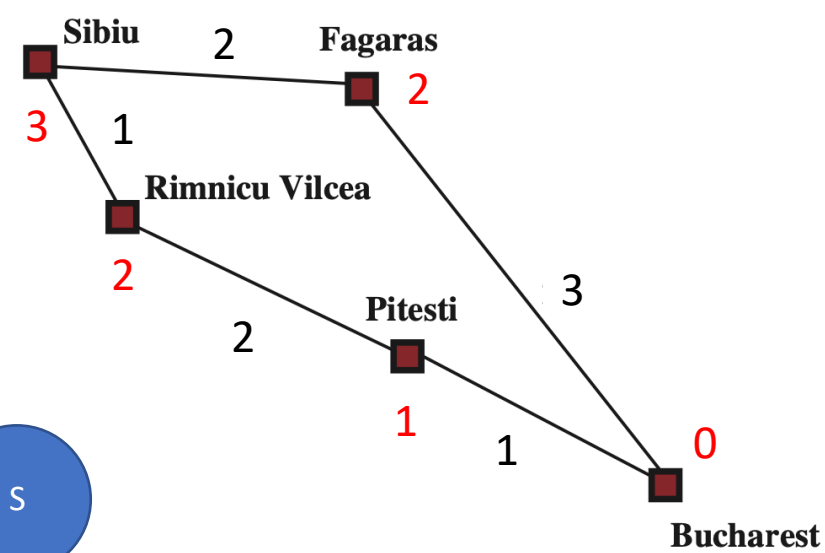
```
return failure
```

$f(n) = h(n)$, only using the estimate of how good a state is



Suboptimal path: $2+3 = 5$

Optimal path: $S - R - P - B: 1+2+1=4$



A* Search

Evaluation function

create **frontier** : **priority queue** ($f(n)$)

insert **Node**(initial_state) to **frontier**

while **frontier** is not empty:

node = **frontier.pop**()

if **node.state** is goal: return solution

for **action** in **actions**(**node.state**):

next_state = **transition**(**node.state**, **action**)

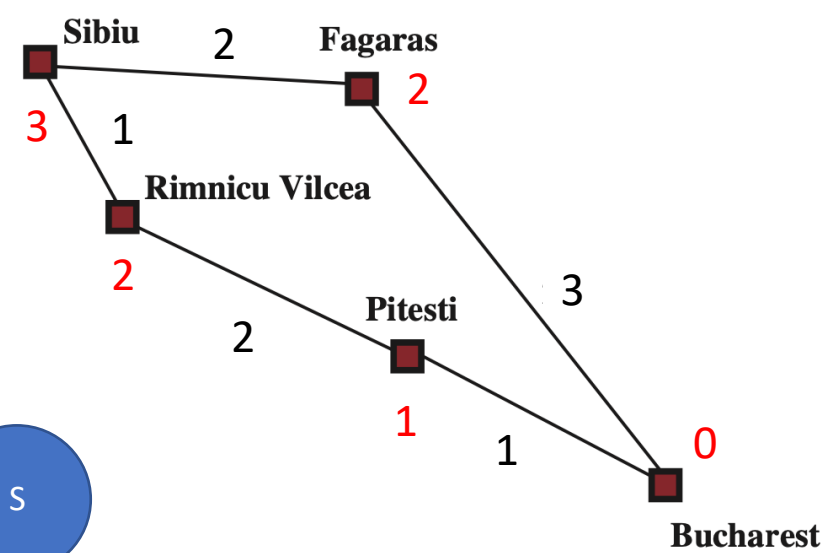
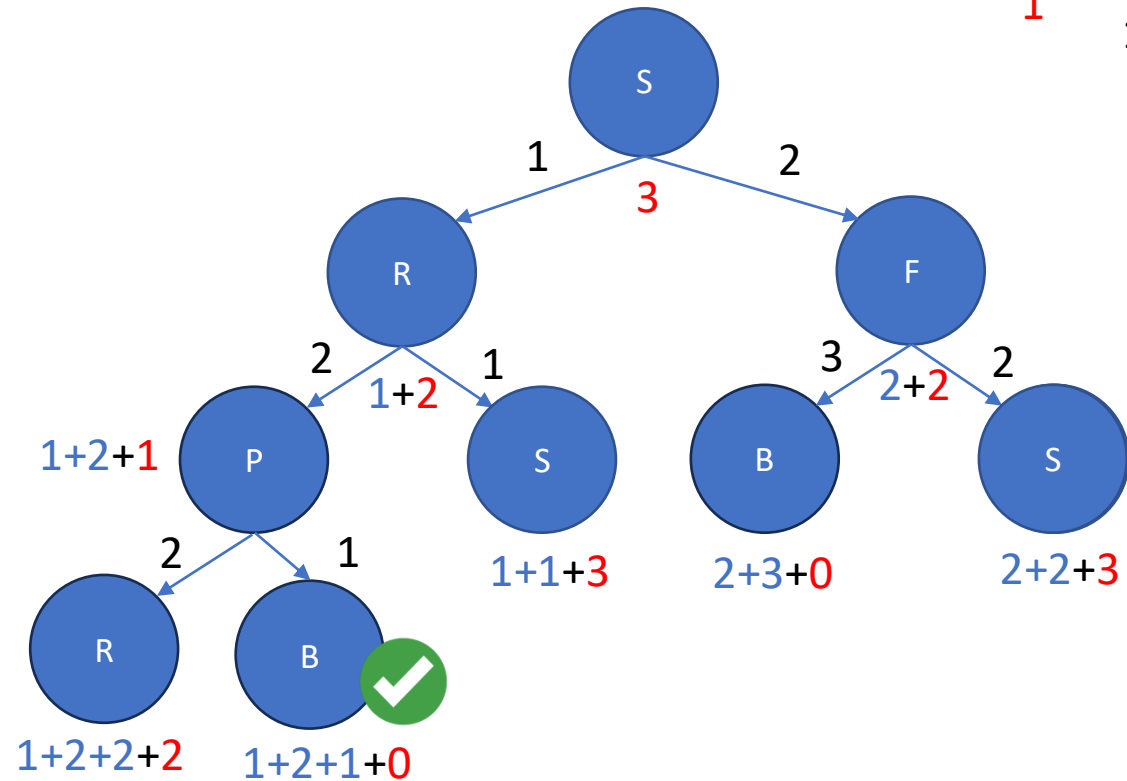
frontier.add(**Node**(**next_state**))

return failure

$$f(n) = g(n) + h(n)$$

Cost to reach n

Heuristic: estimated cost from the state at node n to a goal

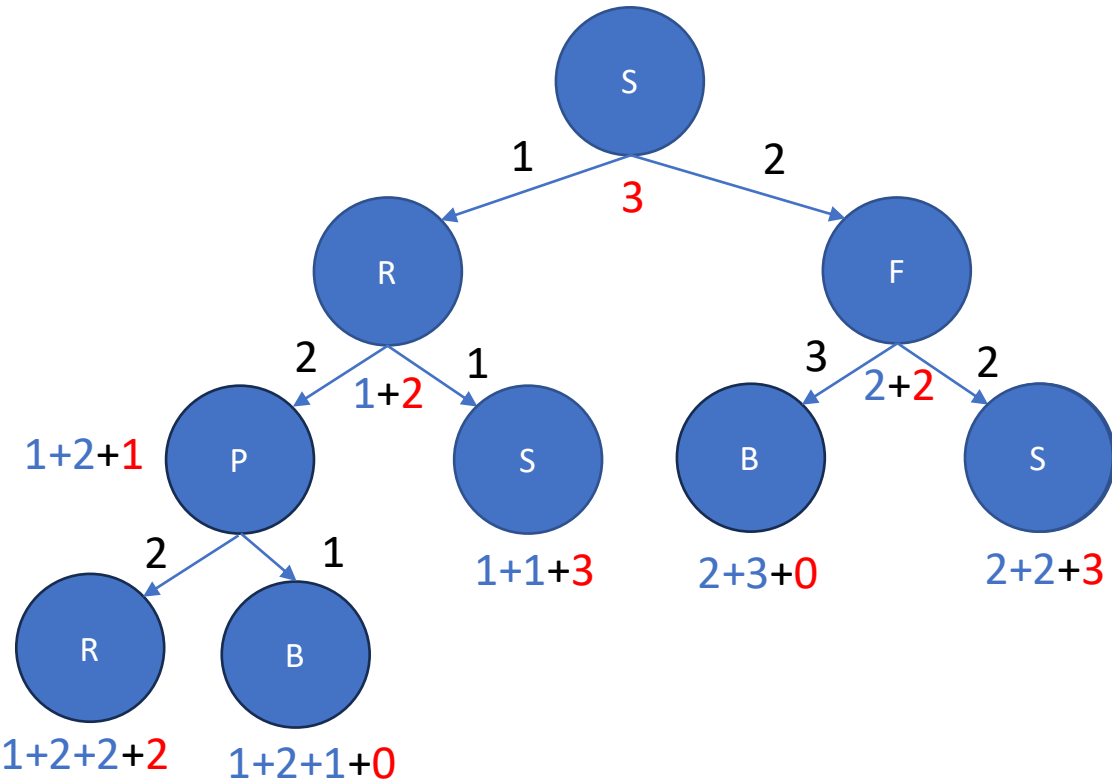


A* Search - Analysis

$$f(n) = g(n) + h(n)$$

Cost to reach n

Heuristic: estimated cost from the state at node n to a goal



Time complexity: (# nodes generated)

Exponential, good heuristic can improve

Space complexity: (size of frontier)

Exponential

Theorem (Completeness): Complete, it will find a goal, if:

- **From UCS:** Assume finite branching factor, all action costs satisfy $c(n, a, n') \geq 0$, for all node n , action a , and next node n' , the optimal solution cost is finite $C^* < \infty$,
- **Additional conditions for heuristic** (see next slides).

Theorem (Optimality): Returns an optimal solution of cost C^* , if:

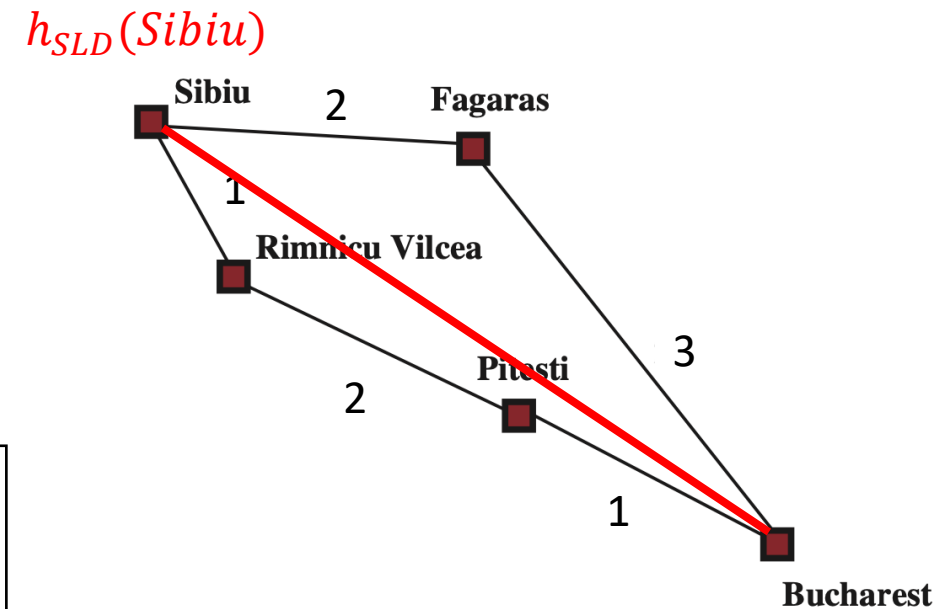
- **From UCS:** All action costs are nonnegative, $c(n, a, n') \geq 0$,
- **Additional conditions for heuristic** (see next slides).

Admissible Heuristics

A **heuristic** $h(n)$ is **admissible** if for every node n , $0 \leq h(n) \leq h^*(n)$, where $h^*(n)$ is the **optimal path cost** to reach the goal state from n .

An admissible heuristic **never over-estimates** the cost to reach the goal.

Theorem (Hart, Nilsson and Raphael, 1968):
If $h(n)$ is admissible, A* **search without visited memory** is **complete** and **optimal** under the conditions stated earlier.



Example: $h_{SLD}(n)$ never overestimates the actual road distance

Consistent Heuristics

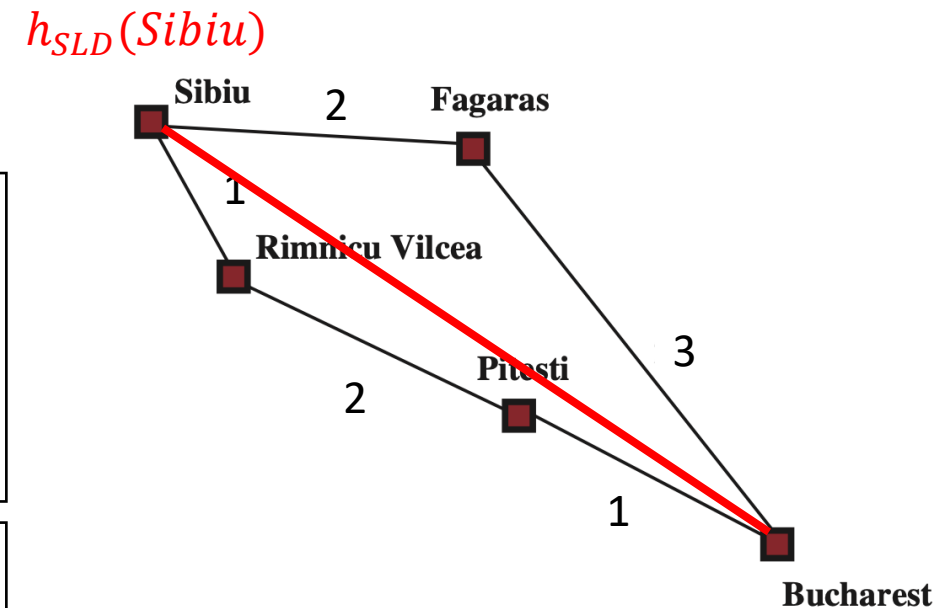
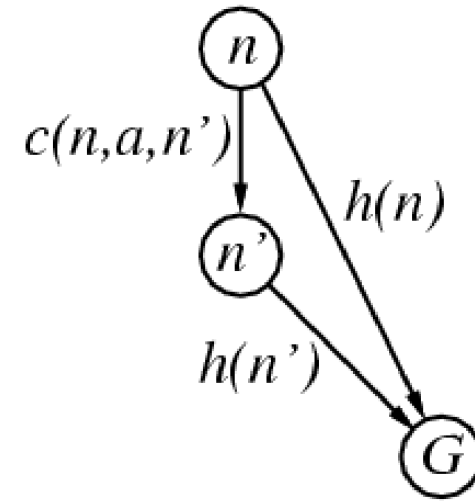
A **heuristic $h(n)$** is **consistent** if for every node n , every successor n' of n generated by any action a , $h(n) \leq c(n,a,n') + h(n')$.

Theorem (Hart, Nilsson and Raphael, 1968):

If $h(n)$ is consistent, A* **search with or without visited memory** is **complete** and **optimal** under the conditions stated earlier.

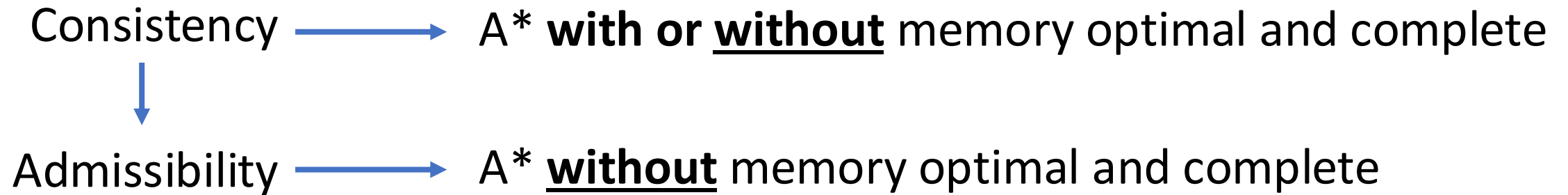
Theorem: Consistency implies admissibility, i.e., if $h(n)$ is consistent then $h(n)$ is admissible.

Proof of consistency $\rightarrow$ admissibility will be discussed in Tutorial 1

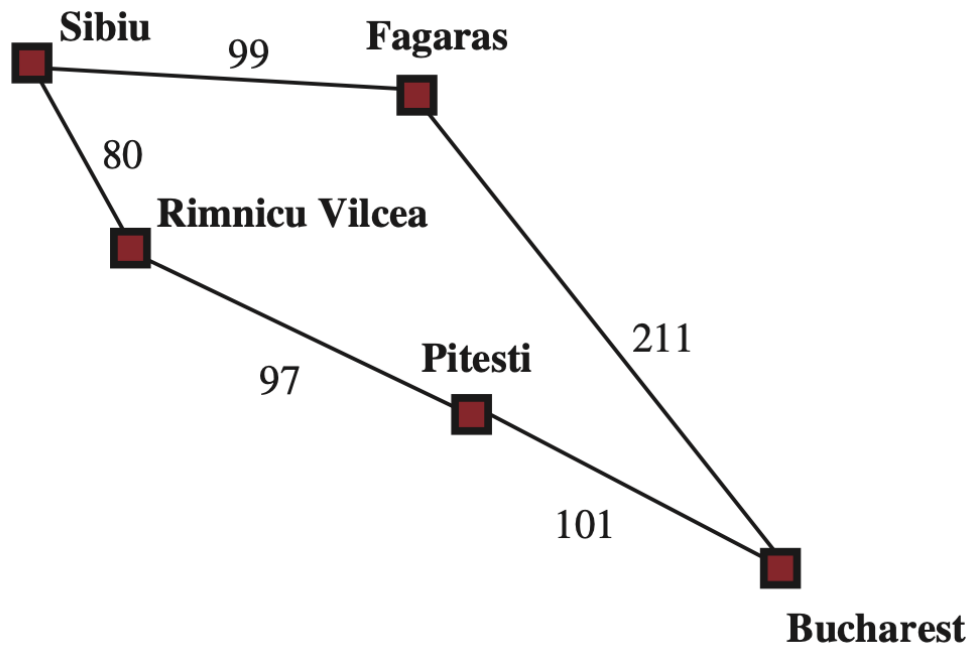


Example: $h_{SLD}(n)$ is consistent

Summary of Heuristic Theorems



“Inventing” Heuristics



- Relaxed problems:
 - A problem with fewer restrictions on the actions is called a **relaxed problem**.
 - This optimal path cost of a relaxed problem is an **admissible heuristic** for the original problem.
- Pattern databases / Precomputations:
 - Store the exact optimal solution costs for every/some subproblems.
 - E.g., store the optimal solution cost of reaching Bucharest from nearby cities.
- Learning from experience:
 - Use machine learning (later in the course).

Relaxed Problems Examples



Original:

An agent can move from point A to B **if**

- A and B are neighbour **and**
- there is a road connecting them

Relaxations:

An agent can move from point A to B **if**

- A and B are neighbour

→ h_1 straight line distance

An agent can move from point A to B

→ $h_2 = 1, h_2 = 0$ if $A=B$

Dominance



If $h_1(n) \geq h_2(n)$ for all n , then h_1 dominates h_2 .

if admissible ($h_1 \leq h^*$), h_1 is better for search .

Reason: h_1 is more accurate as it is closer to h^* (actual cost)

Example:

- h_1 straight-line distance
- $h_2 = 1, h_2 = 0$ if $A=B$

h_1 dominates h_2

Outline

- Designing an agent for search problems
- Search algorithms primer
- Uninformed search
 - Breadth-first, depth-first, and uniform-cost search
- Informed search
 - A* search
 - Heuristics
- **Search strategies**
 - Depth-limited and iterative-deepening search

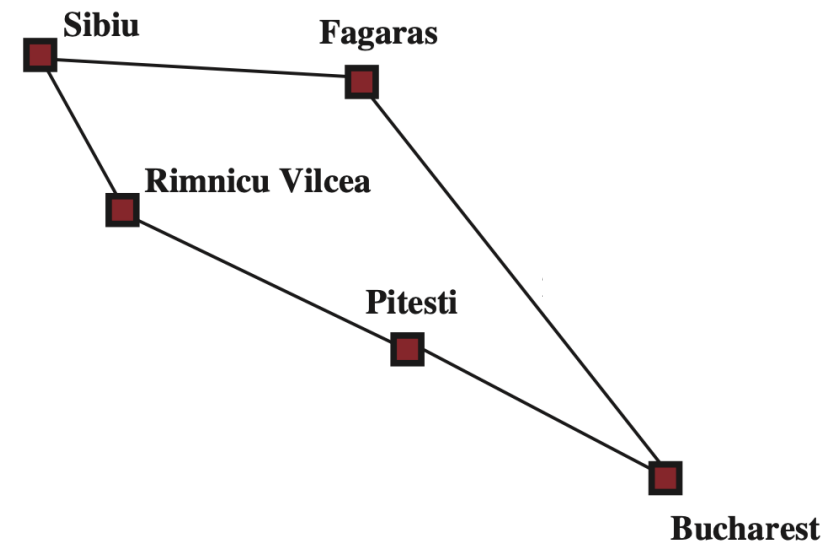
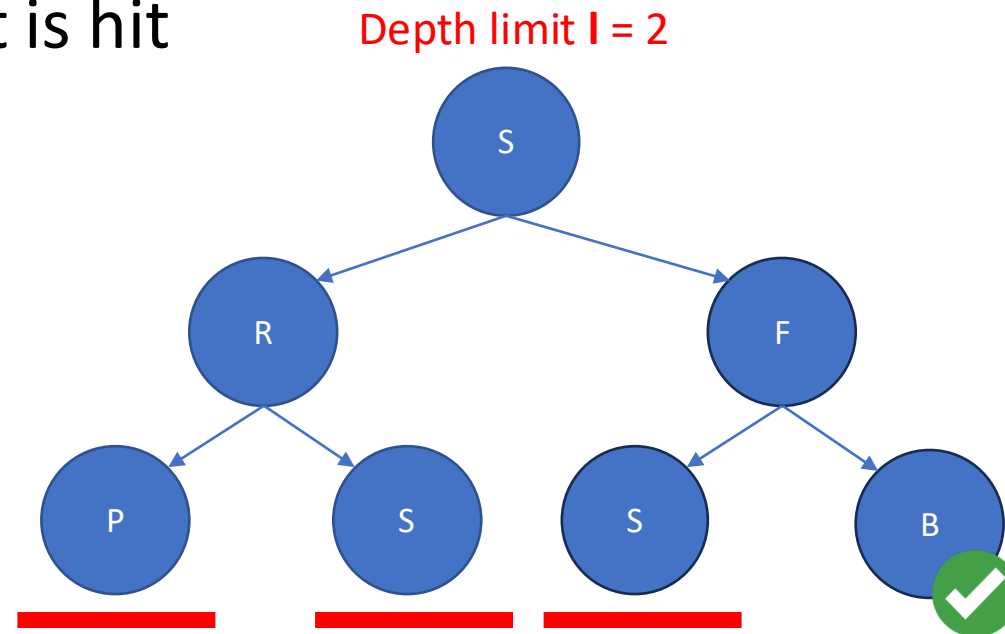
Search Strategies

- Without visited memory, search algorithms **may not terminate**.
 - Depth-first search (frontier: stack). Uses polynomial memory but may not terminate. It is also not optimal (even if we use visited memory).
 - On problems where solution does not exist, search algorithms do not terminate and keep on trying to find a solution.
- A general strategy that we can use is to **set a strict limit on the resources** the algorithm can use.
 - Example: set the depth limit
- Now, we will see how this strategy gives birth to a search with **polynomial memory** like DFS, but it is **complete** and **optimal**.

Depth-limited Search (DLS)

Limit the search to depth l

Backtrack when limit is hit



Depth-limited Search (DLS)

Limit the search to depth l

Backtrack when limit is hit

Time complexity: (# nodes generated)

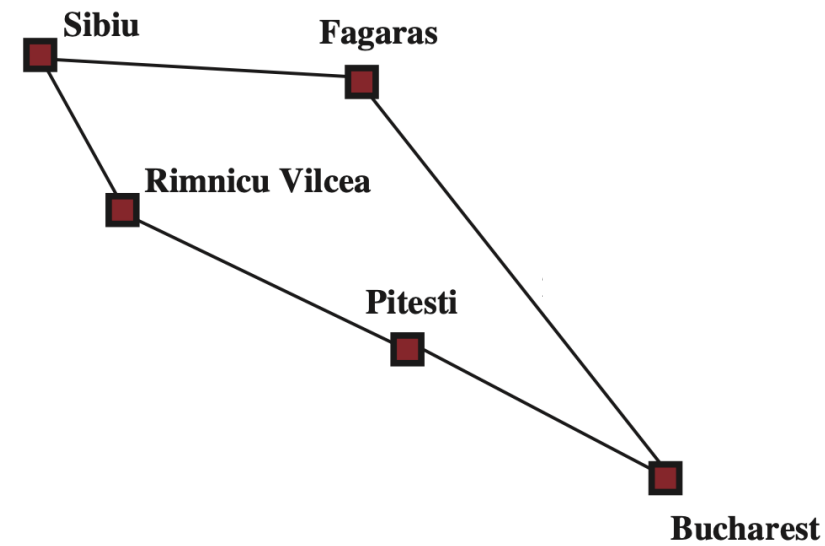
Exponential w.r.t. the depth limit

Space complexity: (size of frontier)

Polynomial w.r.t. the depth limit if used with DFS

Complete: No

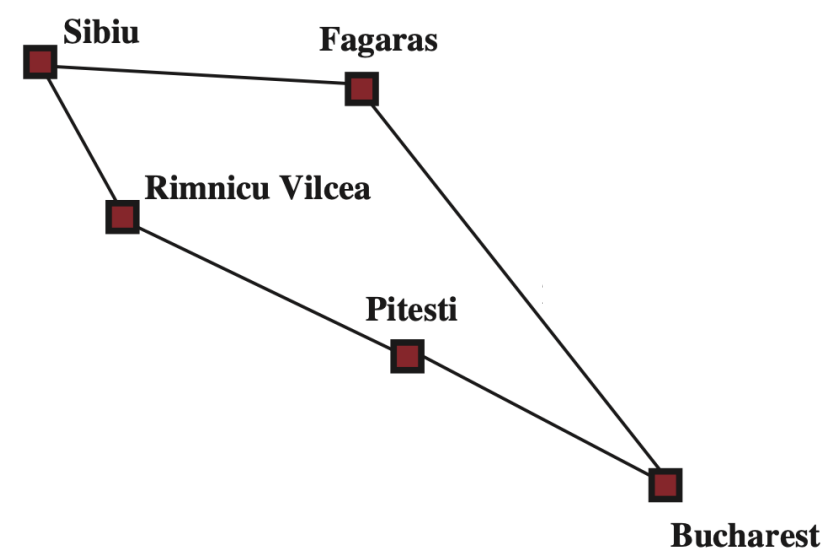
Optimal: No, if used with DFS



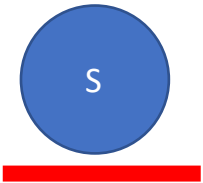
Iterative Deepening Search (IDS)

Search with depth limit 0, ..., ∞

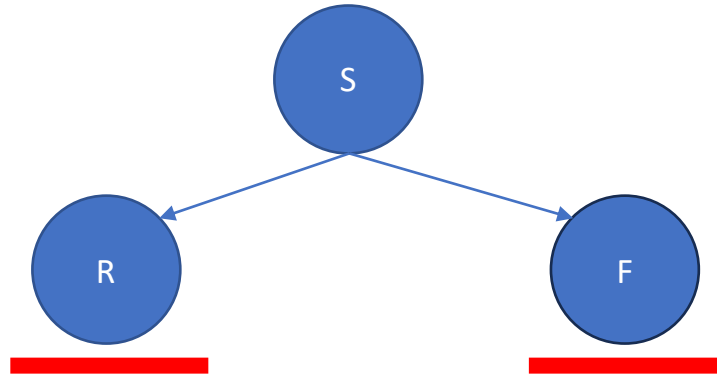
Return solution if found



Depth limit = 0

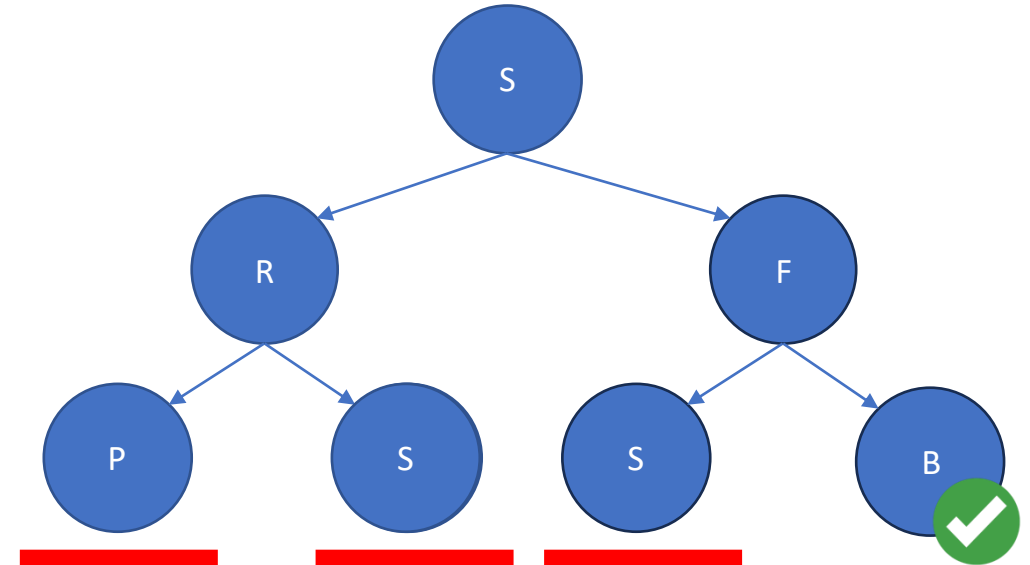


Depth limit = 1



Overhead!

Depth limit = 2



Iterative Deepening Search (IDS)

Search with depth limit $0, \dots, \infty$

Return solution if found

Time complexity: (# nodes generated)

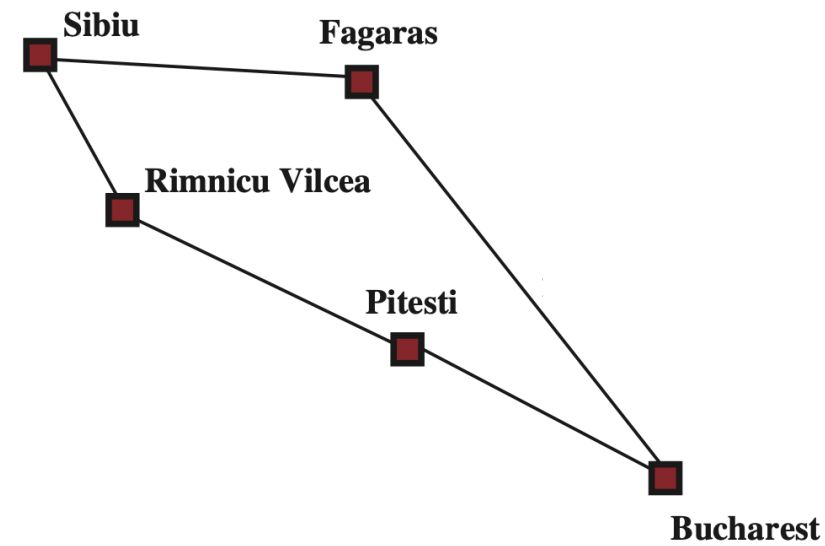
Exponential w.r.t. the depth of optimal solution (with overhead)

Space complexity: (size of frontier)

Polynomial if used with DFS

Complete: *Yes*, under the same conditions that guarantee BFS completeness.

Optimal: *Yes*, under the same conditions that guarantee BFS optimality.



Further Reading (Optional)

- Full analysis of BFS, UCS, DFS, DLS, and IDS (AIAMA 3.4)
- Backward Search (AIAMA 3.4.6)
- Bidirectional Search (AIAMA 3.4.6) (Hart, Nilsson and Raphael, 1968)
- Proof: admissible heuristic → optimal search without visited memory
- Proof: consistent heuristic → optimal search with visited memory (AIAMA 3.5.2)
- Greedy best-first search (AIAMA 3.5.1)
- Variants of A* search (AIAMA 3.5.3)
 - Iterative Deepening A* (IDA*)
 - Simplified Memory-bounded A* (SMA*)

Summary

- Problem solving agents:
 - Goal formulation -> problem formulation -> search -> execute
- Uninformed search
 - Breadth-first search (BFS) – queue
 - Uniform-cost Search (UCS) – priority queue (cost so far)
 - Depth-first Search (DFS) – stack
- Informed search
 - A* Search – priority queue (cost so far + heuristic)
 - Heuristics:
 - Admissible – never overestimate
 - Consistent – triangle inequality
 - Dominant – $h_1(n) \geq h_2(n)$ for all n
 - Inventing heuristics: relaxed problems, pattern databases, learning
- Search strategies
 - Depth-limited & Iterative deepening search (DLS & IDS)

Coming Up Next Week

- Local search
 - Hill climbing
 - ...
- Adversarial search
 - Games
 - Minimax
 - Alpha-beta pruning
 - ...

To Do

- **Lecture Training 2**
 - +550 EXP
 - +100 Early bird bonus
- **Problem Set 1**

Will be released later today!

Appendix

Summary: Uninformed Search Algorithms

- Search algorithms
 - Breadth-first search – **queue**, explore layer by layer
 - Uniform-cost search – **priority queue** (path cost), explore “tier” by “tier”
 - Depth-first search – **stack**, go deep first then backtrack

Summary: Informed Search Algorithms

- Informed search: guide search with extra information
- Best-first search
 - A* search
 - $f(n) = g(n) + h(n)$, cost so far + heuristic
- Heuristics
 - Admissible: $h(n) \leq h^*(n)$
 - Consistent: $h(n) \leq c(n,a,n') + h(n')$
 - Dominant: if $h_1(n) \leq h_2(n)$, h_2 dominant
- Creating admissible heuristic: **optimal path cost of the relaxed problem**

Summary: Search Strategies

- Depth limited search – **limit max depth** of the search
- Iterative deepening search – try DLS with depth limit $0, \dots, \infty$

Summary: Search Algorithms

Worst-case

Name	Time Complexity ¹	Space Complexity ¹	Complete?	Optimal?
Breadth-first Search	Exponential	Exponential	Yes	Yes
Uniform-cost Search	Exponential	Exponential	Yes ³	Yes ³
Depth-first Search	Exponential	Polynomial	No Yes ⁴	No
Depth-limited Search	Exponential	Polynomial ²	No	No ²
Iterative Deepening Search	Exponential	Polynomial ²	Yes	Yes
A* Search	Exponential	Exponential	Yes ³	Yes if admissible
A* Search with visited memory	Exponential	Exponential	Yes ³	Yes if consistent

- 1) In terms of some notion of depth/tier
- 2) If used with DFS
- 3) If edge costs are positive
- 4) With visited memory